abyss

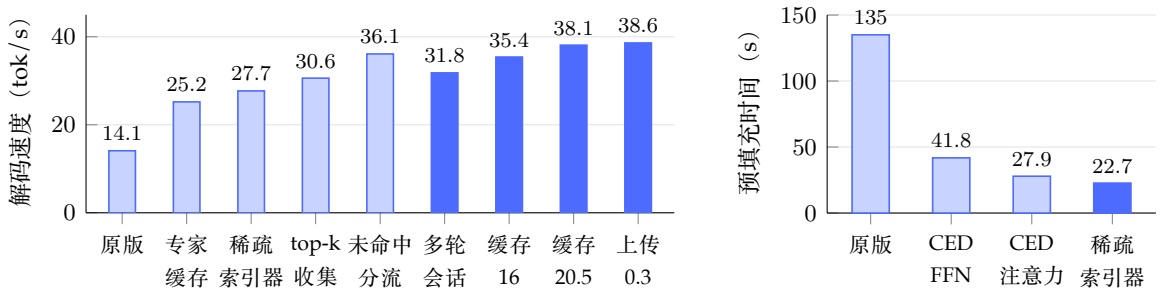
Abyss：在两张显卡上分层运行 DeepSeek-V4.1-Flash

Abyss

2026 年 10 月

摘要

长上下文智能体把推理的压力从算力推到了显存、内存和 PCIe 上。DeepSeek-V4.1-Flash 总参数 763B，原生权重约 474 GiB，超过一台双卡工作站（2 × RTX PRO 6000 Max-Q，显存合计 191 GiB；内存 192 GB）的全部容量。我们在 mx-llama.cpp 上实现了一个分层推理调度器 Abyss：编码器各层的专家常驻显存，解码器后 18 层的专家放在主机内存，由 GPU 上的专家缓存和 CPU 分担，Engram 大表留在 NVMe。围绕这款模型的编码器-解码器结构（CED）、压缩注意力和稀疏索引器，我们依次实现了 CED 感知预填充、Engram 预取、官方稀疏索引器、解码注意力的 top-k 收集、按热度准入的专家缓存未命中分流、预填充大批次的 CPU 分流，以及让提示词末段一遍算完的调用合并；把内存从 3600 MT/s 提到 5200 MT/s；再修掉计算图的两处显存浪费、让索引器按查询分块，把腾出的显存和 GPU0 的空余显存都给专家缓存（12 GB 增到 20.5 GB），并让预填充的大批次直接用缓存里的专家。在保持原生 MXFP4 精度的前提下（每项改动对改动前的 KLD 都在数值底线以内），预填充的提升最大：4.8 万 token 提示词从 135 s（355 tok/s）降到 22.7 s（2,111 tok/s），快约 6 倍，其中 CED 感知预填充把解码器层的计算量从约 96 万个「token × 层」降到约 2.4 万个，贡献最大；Engram 页缓存冷时，预取把冷读开销从约 39 s 降到约 2 s；多轮智能体会话里，之后每轮（1,100–1,600 token）的预填充从 5.7–7.2 s 降到 2.3–2.9 s。解码从 4.8 万上下文的 14.1 tok/s 提升到只追加的多轮会话里的约 38 tok/s（9.6 万上下文；生产上真实会话的平均是 36.8 tok/s），上下文也从 12.8 万扩到了 19.2 万；剩下的时间主要花在把未命中的专家从内存经 PCIe 搬到 GPU，以及等 CPU 算专家。我们还按官方 `encoding.py` 重写了对话模板，消除了智能体会话里思考越写越重复的问题，并按官方 Recipe 的规则编码请求文字里的控制 token；把权重换成 DeepSeek 10 月 17 日更新的 1017 版：用自己写的转换脚本按现有 GGUF 的格式转换（在原版权重上抽样，与现有 GGUF 逐字节一致），同一语料上 PPL 低 3.4%，速度和显存不变。我们同时记录了评估方法（两档数值底线、对官方实现的单层对拍、逐 token 剖析，以及和 llama-bench、AIPerf 的口径差别）、一个让解码速度虚高的测法陷阱、两条测过但没有采用的路（每 token 少用一个专家、DSpark 投机解码），以及在真实 Codex 和 Hermes 负载中发现并修复的缺陷。



(a) (b)

图 1 | (a) 单流解码速度，依次叠加各项优化（第 4 节）。前 5 根是 4.8 万上下文，其中第 3–5 根是同一提示词第二次生成，缓存命中偏高；后 4 根是只追加的多轮会话实测（9.6 万上下文、内存 5200 MT/s，§4.4、§4.5），依次是 10-25 的配置，索引器分块后专家缓存加到 16 GB，借用 GPU0 的空余显存后加到 20.5 GB，上传比例改为 0.3。(b) 4.8 万 token 提示词的预填充时间，Engram 页缓存已热（第 5 节）。

目录

1 引言 4

2 硬件与模型 6

2.1 硬件 6

2.2 模型 6

2.3 权重格式与量化 8

2.4 方案 B 的布局 8

3 评估方法 9

3.1 数值底线 9

3.2 对官方实现的对拍 9

3.3 剖析 11

3.4 标准口径 11

4 解码 12

4.1 GPU 专家缓存 12

4.2 解码注意力只取 top-k 行 12

4.3 按热度准入的未命中分流 13

4.4 实际会话中的解码速度 13

4.5 腾出显存给专家缓存 14

4.6 剩下的时间 15

4.7 测过但没有采用的两条路 16

5 预填充 16

5.1 CED 感知预填充 17

5.2 Engram 预取 17

5.3 补上官方稀疏索引器 17
5.4 大批次分流 18
5.5 末段一遍算完 19
5.6 大批次用缓存 20

6 服务与智能体负载 21
6.1 多轮会话的缓存复用 21
6.2 prompt 缓存的选择 21
6.3 真实负载暴露的缺陷 21
6.4 重启后的长对话首次预填充 22
6.5 对话模板与采样 23
6.6 请求文字里的控制 token 23
6.7 槽位被只共享开头的请求占用 24
6.8 智能体客户端的额外请求 24

7 把内存提到 5200 MT/s 24

8 把上下文扩到 19.2 万 25

9 换用 1017 权重 26

10 结论、局限与未来方向 28

A 补丁清单 31

B 复现 31

C 术语表 33

1 引言

智能体把大模型的典型请求从「短问短答」变成了「长上下文、多轮、带工具调用」。一个编程智能体每一轮都要在几万 token 的上下文后面追加回复、工具结果或追问，再让模型继续生成。这种负载有两个特点：一是上下文长，注意力和 KV 的开销随之增长；二是每一轮新增的内容不多，但首 token 之前的那段预填充决定了用户等待的时间。

DeepSeek-V4.1-Flash [1] 是为这类负载设计的。它有 552B 主干参数、196.6B 的 Engram 查表和约 14B 的 DSpark 草稿模型；40 层中前 20 层是因果编码器、后 20 层是解码器（CED），提示词的每个 token 只需走完编码器，生成的 token 才走完全部 40 层；注意力把 KV 按 2 倍或 1 倍压缩，再由稀疏索引器为每个查询挑出 512 个压缩位置；路由专家 384 个，每个 token 选 6 个，权重是原生 MXFP4。

问题在于容量。原生权重约 474 GiB（专家 268.9 GiB，Engram 189 GiB），而我们的机器只有两张 RTX PRO 6000 Max-Q（每张 95.6 GiB）和 192 GB 内存，合计约 383 GiB。公开的双卡方案 [3] 把专家量化到 3.0 bpw，速度很快（同样两张 RTX PRO 6000，单流解码 66–74 tok/s，4.6 万 token 提示词预填充约 1 万 tok/s），但相对原版的 KL 散度为 0.0545、top-1 一致率 92.9%，而且需要约 250 GiB 空闲内存，本机放不下。我们选择了另一条路：保留原生 MXFP4，把权重按热度分布在显存、内存和 NVMe 三层（方案 B），在 mx-llama.cpp [2] 上实现调度器 Abyss，再逐项找出瓶颈并优化。

本报告记录这一过程。主要结果见表 1：

- **解码**：GPU 专家缓存、解码注意力只取 top-k 行、按热度准入的未命中分流，使解码从 14.1 tok/s 提升到只追加的多轮会话里约 30 tok/s（内存 3600 MT/s），4.8 万和 9.6 万上下文一样；内存提到 5200 MT/s 后为 31–33 tok/s；再把计算图和索引器腾出的显存、GPU0 的空余显存都给专家缓存后约 38 tok/s，生产上真实会话的平均是 36.8 tok/s。
- **预填充**：CED 感知预填充让解码器层只计算后面还能看到的 token，加上 Engram 预取和官方稀疏索引器，4.8 万 token 提示词从 135 s 降到 22.7 s；大批次分流、末段一遍算完和大批次用缓存再把多轮会话每轮（1,100–1,600 token）的预填充从 5.7–7.2 s 降到 2.3–2.9 s。
- **测法**：同一提示词第二次、一字不差的生成会让专家缓存命中偏高，解码速度虚高约 15–20%。我们用只追加的多轮会话重新测量，并据此修正了此前的结论；也给出了 llama-bench 和 AIPerf 的结果，并说明它们为什么和我们的测法不同（§3.4）。
- **正确性**：所有改动都有对拍。我们区分了两档数值底线，并对稀疏索引器和解码注意力做了对官方 PyTorch 实现、对 float64 的逐层对拍。
- **真实负载**：在 Codex 和 Hermes 的实际请求中发现了 prompt 缓存的选择问题、槽位被只共享开头的请求占用、请求文字里的控制 token 字样被当真，以及三处只在特定提示词长度下出现的崩溃，均已修复。

表 1 | 各阶段的改动与效果。解码在 4.8 万上下文下测（10-28 的几行是 9.6 万上下文、只追加的多轮会话）；预填充为 4.8 万 token 提示词、Engram 页缓存已热（最后一行是多轮会话的每轮预填充）；KLD 是对改动前的 logits，「—」表示没有改变数值。

补丁	改动	效果	对拍
环境变量	预填充的 MXFP4 矩阵乘改用 8 bit 激活（W4A8）	预填充约慢 5%	0.044 → 0.012*
分支自带	GPU1 上的专家缓存（14 GB，约 780 槽位，LRU）	解码 14.1 → 25.2 tok/s	0.0005
0003	CED 感知预填充：解码器层只算可见 token 的 FFN	预填充 135 → 41.8 s	0.00004
0004	Engram 整批预取	冷缓存 91.1 → 54.3 s	—
0005	解码器层的注意力也只算子集	41.8 → 27.9 s	0.00007
0006	补上官方稀疏索引器（top-512）	27.9 → 22.7 s；修正长上下文	见 §5.3
0008	解码注意力只取 top-k 行	解码 27.7 → 30.6 tok/s‡	0.003†
0009	未命中按热度一半上传、一半交给 CPU	30.6 → 36.1 tok/s‡	0.0003†
0014	预填充大批次分流	每轮 5.7–7.2 → 4.6–5.6 s	0.0005
0015	提示词末段一遍算完：末段不拆调用，最后一个 ubatch 对齐到满	每轮 4.6–5.6 → 2.9–3.5 s	0.009§
BIOS	内存 3600 → 5200 MT/s	解码约 +6%，每轮 2.7–3.2 s	重跑一致
配置	上下文 12.8 万 → 19.2 万，专家缓存 14 → 12 GB	解码约 −3%，预填充不变	0.000005
模板	对话模板按官方 `encoding.py` 重写，采样改为官方推荐值	思考不再越写越重复	—
0017	槽位被只共享开头的请求占用前先存进内存缓存	下一轮重算 26,438 → 30 token	输出相同
权重	换用 1017 版（自己转换）	PPL −3.4%，速度不变	0.075¶
0018	请求文字里的控制 token 字样只按普通文字编码（官方 Recipe 的规则）	不再被当成控制 token	逐 token 一致**
0019	全零张量用到时才分配，没有查询行的层不算查询	GPU0 少用 2.6 GB；预填充快 3–5%	输出不变
0020	索引器和预填充注意力按查询分块，专家缓存 12 → 16 GB	解码 31.95 → 35.42 tok/s	0.0079
配置	第 12 层的 gate、up 专家放到 GPU0，专家缓存 16 → 20.5 GB	解码 35.54 → 38.06 tok/s	输出相同
配置	未命中的上传比例 0.5 → 0.3	解码 37.75 → 38.64 tok/s	约 2e-4†
0023	预填充大批次里缓存已有的专家直接在 GPU1 上算	每轮快 0.3–0.4 s	1.7e-4†

* 预填充路径对解码路径的 KLD，不是对改动前。† server 上 temperature 0 生成时的 top-20 KL；0008 经过编码器层，属于较高的一档底线（§3.1）。‡ 同一提示词第二次、一字不差的生成，缓存命中偏高；只追加的多轮会话见 §4.4。§ 和只换 ubatch 大小的差别一样（§5.5）。¶ 衡量的是权重的变化，不是数值误差（§9）。** 和官方编码器逐 token 一致；不含这类字样时和原编码相同（§6.6）。

2 硬件与模型

2.1 硬件

表 2 是机器的配置和实测带宽。两点决定了后面的设计：内存是双通道、每通道两条，优化过程中运行在 3600 MT/s，纯读只有约 44 GB/s（最后提到 5200 MT/s，见第 7 节）；两张卡各走 PCIe 5.0 x8，单卡主机到显存约 28.9 GB/s，但两卡同时从主机读合计只有 43.4 GB/s，被内存带宽卡住。所以无论专家由 CPU 计算还是上传到 GPU，每读一个专家都要占用同一份内存带宽。

表 2 | 硬件与实测带宽（2026-10-20，STREAM 与 nvbandwidth；括号里是 2026-10-25 内存提到 5200 MT/s 后的值）。

项目	配置与实测
GPU	2 × RTX PRO 6000 Blackwell Max-Q（300 W），每张 97,887 MiB GDDR7，SM120
互联	PCIe 5.0 x8 / x8，P2P 经 CPU，无 NVLink；卡间单向约 27.7 GB/s
CPU	AMD Ryzen 9 9950X，16 核 32 线程，单 NUMA
内存	4 × 48 GB DDR5（双 rank），双通道 2DPC，3600 MT/s；纯读 44 GB/s，STREAM Copy 42 / Triad 32 GB/s（5200 MT/s：54、53.5 / 39.7）
主机 → 显存	单卡 28.9 GB/s；两卡同时合计 43.4 GB/s（5200 MT/s：50.6）
存储	Samsung 9100 PRO 4TB（PCIe 5.0）、990 PRO 4TB（PCIe 4.0）

2.2 模型

图 2 是 DeepSeek-V4.1-Flash 的逐层结构和方案 B 在本机的布局，维度取自 GGUF 元数据。与本报告相关的结构特点有四个：

- **编码器-解码器（CED）**。第 0–19 层是因果编码器，第 20 层的压缩器生成所有解码器层共享的全局 KV。解码器层只通过各自 128 个 token 的滑动窗口读自己的隐状态，因此提示词前面的大部分 token 不需要走完解码器。
- **压缩注意力**。第 0–1 层只有滑动窗口；第 2–19 层把每 2 个 token 压成一行，第 20–39 层每个 token 一行。压缩 KV 只在第 2、8、14、20 层写入，其后的层复用。每个查询有 64 个头，键值只有 1 个 512 维的头（K = V）。
- **稀疏索引器**。第 2、8、14、20、24、28、32、36 层用 32 个头、128 维、FP4 的查询和键给压缩位置打分，选出 top-512；第 20 层还按每 8 行一块挑出 2,048 个候选块，供第 24–36 层重选。其余层沿用最近索引源的结果。
- **专家与 Engram**。每层 384 个路由专家选 6 个，外加 1 个共享专家；每个 MXFP4 专家约 18.8 MB，每层 7.2 GB。第 1、14 层按最近 2–4 个 token 的 n-gram 哈希查两张约 3.84 亿行的大表。

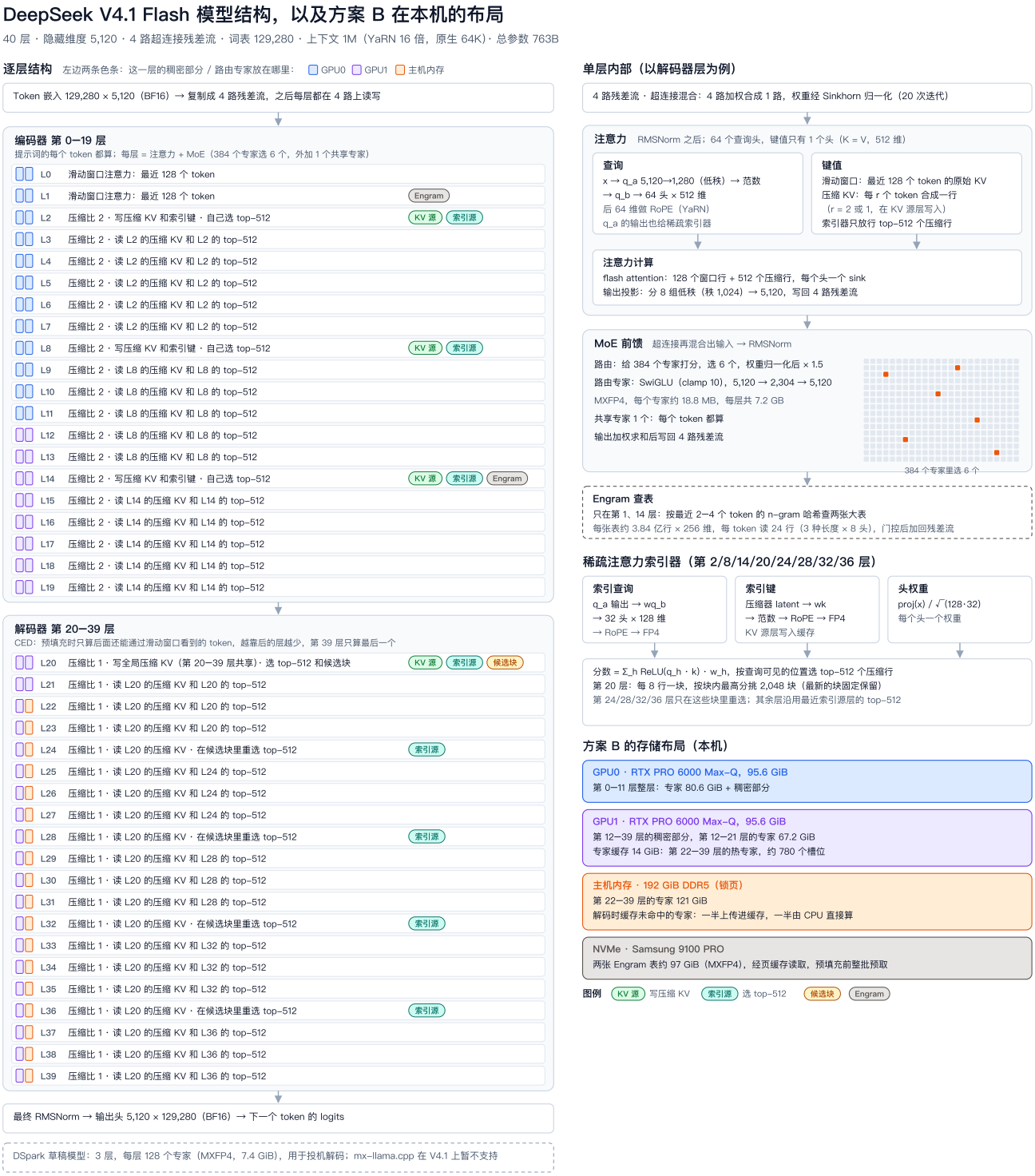


图 2 | DeepSeek-V4.1-Flash 的逐层结构，以及方案 B 在本机的布局。左侧每行一层，色条表示该层稠密部分和路由专家所在的位置；右侧是单层内部、稀疏索引器和存储布局。

2.3 权重格式与量化

本报告的优化都没有对模型再做量化：改动的只是权重放在哪一层，以及哪些 token、哪些行不必计算，每项改动都和改动前对拍（§3.1）。不过我们运行的 GGUF [2] 本身和官方 checkpoint 不完全相同。表 3 是逐个统计 12 个分片的结果。

表 3 | GGUF 中各类权重的格式，与官方 checkpoint 对比。

权重	官方	GGUF	大小	精度变化
路由专家	FP4（E2M1，每 32 个一个缩放）	MXFP4	275.7 GiB	无，同一种格式
注意力等稠密矩阵	FP8	Q8_0	5.5 GiB	很小，都是 8 bit，只是缩放方式不同
共享专家	FP8*	Q8_0	1.5 GiB	同上
Engram 表	**FP8，约 189 GiB**	**MXFP4**	**97.3 GiB**	**从 8 bit 降到 4 bit**
嵌入、输出、路由门、压缩器、索引器投影	—	BF16	2.8 GiB	无
各种 norm、偏置	—	F32	0.2 GiB	无

* 按官方 `inference/model.py` 推断：只有路由专家用 FP4，其余线性层走 FP8 矩阵乘；共享专家没有单独核对。
—：未单独核对，GGUF 中是 16 位或 32 位浮点。

真正降了精度的只有 Engram 表。转换 GGUF 时它从 FP8 压到了 MXFP4，体积减半，也更容易留在页缓存里。这是转换者的选择，我们直接继承；本报告的对拍都是同一个 GGUF 改动前后的比较，没有直接测过这一步的影响；NVIDIA 在其 NVFP4 版本中做了同样的转换，测得表的相对 L2 误差约 12%，而 GPQA、SciCode 等基准的分数几乎不变 [7]。

计算时还有两处量化。GPU 和 CPU 上的矩阵乘都先把激活按 32 个一组量化成 int8 再乘，即 W4A8；官方参考实现用 FP8 激活，同为 8 bit。fork 在 Blackwell 上的预填充默认是 W4A4（激活只有 4 bit），我们改回了 8 bit（§3.1）。稀疏索引器的查询和键用 FP4，这是官方的设计，我们照官方实现做。KV cache 用默认的 F16，没有量化。

作为对比，方案 A 要把路由专家从约 4.25 bpw 再压到 3.0 bpw，相对原版的 KL 散度为 0.0545 [3]；方案 B 不做这一步，这正是选它的原因。

2.4 方案 B 的布局

方案 B 用层切分（`-sm layer -ts 12/29`）：GPU0 放第 0–11 层整层；GPU1 放第 12–39 层的稠密部分和第 12–21 层的专家；第 22–39 层的专家（121 GiB）放在锁页主机内存；Engram 表留在 NVMe，经页缓存读取。编码器的专家全部在显存，所以只要提示词不走解码器层，预填充就基本不碰主机内存。表 4 列出各层的内容。后来加入的 GPU1 专家缓存占 14 GB，大批次分流让 GPU0 的计算缓冲区多占约 1.9 GB；12.8 万上下文时，两张卡分别只剩约 3.6 GB 和 1.2 GB。上下文扩到 19.2 万后，专家缓存降到 12 GB（§8）；10-28 修掉计算图的两处显存浪费、让索引器分块后加到 16 GB，再用 `-ot` 把第 12 层的 gate、up 专家张量（共约 4.6 GB）放到 GPU0 的空余显存，加到 20.5 GB、约 1,140 个槽位（§4.5）。

图 3 把放置和运行时的分工画在一起。解码时两张卡串行：GPU0 只算前 12 层，每 token

表 4 | 方案 B 的存储布局（最终配置）。

位置	内容	容量
GPU0	第 0–11 层整层（专家 80.6 GiB + 稠密部分），第 12 层的 gate、up 专家	95.6 GiB
GPU1	第 12–39 层稠密部分，第 12–21 层专家（第 12 层只有 down），专家缓存 20.5 GB	95.6 GiB
主机内存	第 22–39 层专家 121 GiB（CUDA_Host 锁页）	192 GB
NVMe	两张 Engram 表（MXFP4，约 97 GiB）	4 TB

约 4 ms；GPU1 算其余 28 层和输出层，还要等未命中的专家从内存上传，是解码的瓶颈，CPU 同时分担一部分未命中的专家。预填充时编码器层的全部 token 都在显存里算，解码器层只算提示词末尾还用得到的 token；第 22–39 层的大批次专家沿用 fork 的做法上传到 GPU0 计算，分到 token 少的专家由 CPU 同时算（§5.4），缓存里已有的直接在 GPU1 上算（§5.6）。

3 评估方法

3.1 数值底线

我们用 llama.cpp 的 KL 散度对拍：先存一份基准 logits，再比较改动后的 logits。早期的编译同一配置重跑完全相同（KLD 为 0）；加了后来的补丁后，重跑之间 KLD 平均约 0、最大约 6e-5，top-1 一致率 100%。重要的发现是数值底线分两档（表 5）：

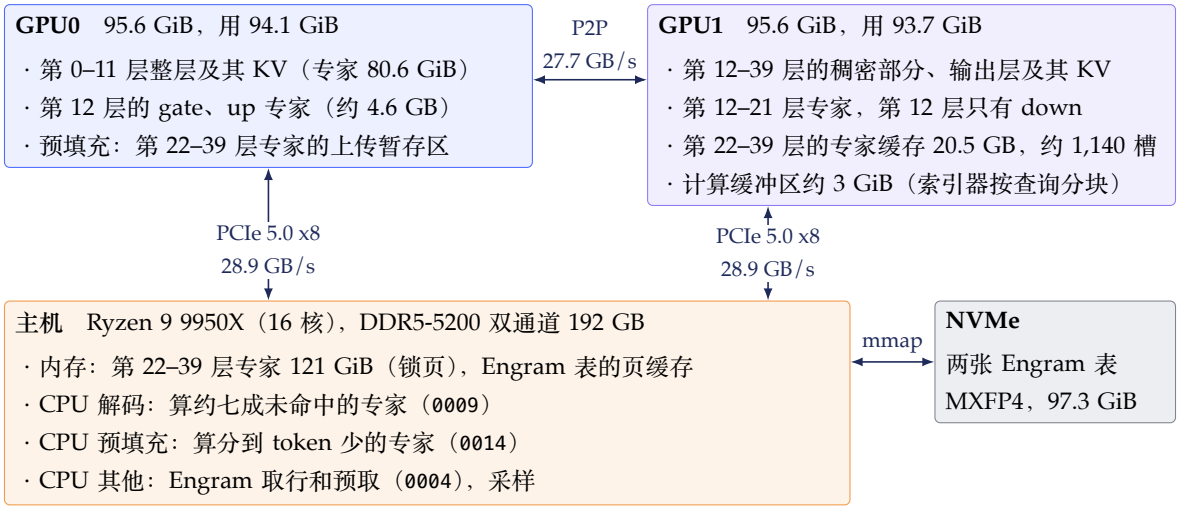
- 改动只落在解码器层（第 20–39 层）时，KLD 约 0.0005。例如把第 22–39 层的专家从 CPU 改到 GPU 上计算。
- 改动经过编码器层（第 0–19 层）时，0.005–0.01 属正常。例如只把第 5 层的专家改到 CPU 计算就是 0.0064；换内核、关融合、换 ubatch 也在这个量级。

原因在 CED：编码器的输出决定了所有位置共享的全局 KV，那里的舍入差别会扩散到之后所有位置；解码器层的差别只影响当前 token。判断一项改动是否正确，必须先确定它落在哪一档。

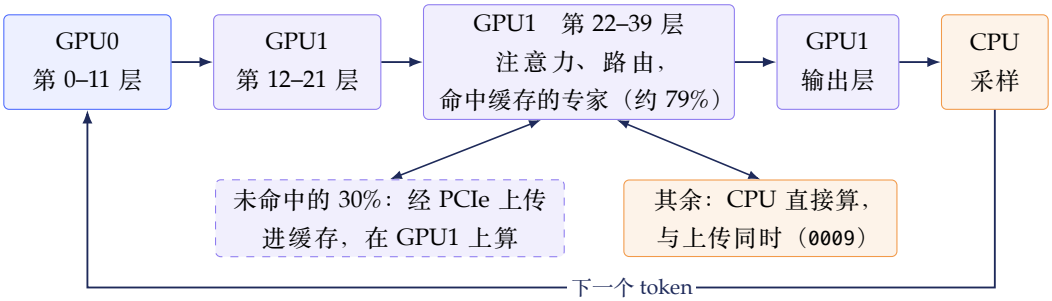
同一套方法一开始就查出了一个精度问题：mx-llama.cpp 在 Blackwell 上对 MXFP4 的预填充默认用 W4A4（激活量化到 4 bit），而官方参考实现是 W4A8。预填充路径对解码路径的 KLD 因此高达 0.044、top-1 一致率 91%；改为 8 bit 激活后降到 0.012、95%，与数值底线相当，预填充只慢约 5%。

3.2 对官方实现的对拍

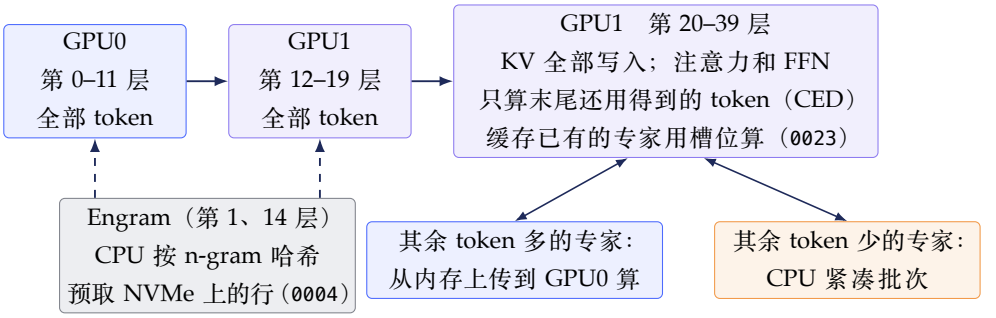
KLD 只能说明最终输出变了多少，不能说明是哪一步造成的。对实现复杂的部分，我们写了独立的参照实现（oracle）逐层对拍：用 `llama-abyss-dump` 导出 fork 计算图里指定的张量，再用 PyTorch 按官方 `inference/model.py` 从同样的输入重算。解码步的注意力还用 float64 重算，单独量出内核的舍入误差。具体结果见 §5.3 和 §4.2。



(a) 放置与链路（19.2 万上下文、专家缓存 20.5 GB 的配置，§4.5）



(b) 解码一个 token：两张卡串行



第 22–39 层、超过 32 个 token 的批次：三路同时算（0014、0023）

(c) 一次预填充调用

图 3 | 方案 B 中 GPU0、GPU1 和主机的分工。蓝色是 GPU0，紫色是 GPU1，橙色是 CPU 和内存，灰色是 NVMe；两卡之间没有 NVLink，P2P 经 CPU 转发。(b) 中两张卡串行，10-28 的生产设置下 9.6 万上下文每 token 约 30 ms：GPU1 上传未命中专家 8.0 ms、GPU1 内核 10.4 ms、GPU0 内核 4.3 ms、两卡都空闲 7.7 ms（§4.6）。(c) 中编码器层算全部 token，解码器层只算末尾子集（§5.1）。

表 5 | 数值底线与 W4A4 问题（2026-10-21）。

比较	改动范围	KLD
同一配置重跑	—	0（后来最大 6e-5）
第 22–39 层专家从 CPU 改到 GPU	只有解码器层	0.0005
第 5 层专家从 GPU 改到 CPU	经过编码器层	0.0064
预填充 W4A4 对解码路径	全部层	0.044
预填充 W4A8 对解码路径	全部层	0.012

3.3 剖析

性能问题一律先剖析再动手。我们用 nsys 抓取一段预填充或一段解码，按设备统计内核、拷贝和空闲时间，再按 token 平均（`tools/eval/nsys_breakdown.py`、`nsys_decode.py`）。多轮智能体用一个模拟 Codex 的脚本测量：首轮带 4.8 万 token 的文档和两个工具，之后每轮回填约 1,500 token 的工具结果或追问，逐轮记录缓存复用和预填充时间（`agent-turns.sh`）。显存用 `GGML_ALLOC_TRACE=1`（`0019`）记下计算图每次分配和释放的张量，列出计算缓冲区峰值时有哪些张量（`alloc-trace.sh`、`alloc_peak.py`）。

3.4 标准口径

为了和其他系统比较，我们也用 llama-bench 和 AIPerf 测了一遍（`std-bench.sh`，生产设置、专家缓存 16 GB，表 6）。两者都和本报告的测法不同。llama-bench 的 pp 每个测试只有一次调用，量到的主要是每次调用把第 22–39 层专家从内存读一遍的固定开销（2,048 token 约 3 s），看不出 CED 对长提示词的加速；它用随机 token，tg 的缓存命中 68–73%。AIPerf 单并发、用编程语料、每次输出 1,024 token：首 token 时间就是新提示词的预填充，每 token 时间和只追加的多轮会话相近。AIPerf 生成的提示词开头相同，不关 prompt 缓存时 12–33% 的 token 直接从缓存取，首 token 时间偏低（9.6 万 token 时 24.3 s），所以请求里要加 `cache_prompt: false`；它的 `--cache-bust` 只支持 chat 接口。

表 6 | 标准口径的测速（2026-10-28，生产设置、专家缓存 16 GB）。llama-bench 的专家缓存大小靠 `0021` 从环境变量读入；AIPerf 为 0.13 版，单并发。

工具	测试	结果
llama-bench	pp512（深度 0 / 3.2 万 / 9.8 万）	约 250 tok/s
	pp2048（同上）	686 / 678 / 631 tok/s
	tg128（同上）	32.5 / 33.0 / 34.3 tok/s
AIPerf	首 token 时间 p50（输入 1K / 8K / 32K / 96K）	2.70 / 5.53 / 13.25 / 37.1 s
	首 token 时间 p99（同上）	2.79 / 5.76 / 15.4 / 41.8 s
	每 token 时间（输出 1,024 token）	29.4–30.8 ms

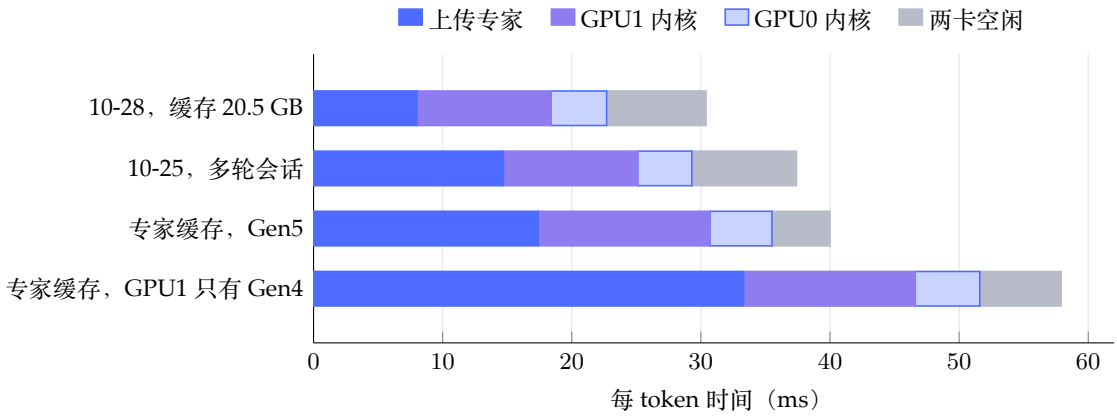


图 4 | 解码的每 token 时间（nsys）。后三行是 4.8 万上下文、内存 3600 MT/s：最下一行是一次开机后 GPU1 的链路只训练到 Gen4，上传慢了一倍（§6.3）；第三行是只开专家缓存时；第二行是全部优化后、只追加的多轮会话中，每 token 37.5 ms（§4.4）。第一行是 10-28 的生产设置（9.6 万上下文、内存 5200 MT/s、专家缓存 20.5 GB），每 token 30.4 ms（§4.6）。

4 解码

方案 B 的原版解码是 16.5 tok/s（空上下文），4 万上下文时约 14.6 tok/s。瓶颈是 CPU：解码器后 18 层每个 token 要读 108 个专家、约 2.0 GB，而内存带宽只有 44 GB/s。

4.1 GPU 专家缓存

mx-llama.cpp 自带一个实验性的专家缓存：在 GPU1 上划出一组槽位（bank），每层一张「专家号 → 槽位号」的映射，命中的专家直接在 GPU 上算，未命中的同步上传并按 LRU 淘汰。我们给它 14 GB（约 780 个槽位，占第 22–39 层专家的 11%）。真实文本的专家局部性很强，LRU 命中率约 75%；4.8 万上下文的解码从 14.1 提升到 25.2 tok/s，短上下文约 29–30 tok/s，KLD 0.0005。

开启缓存后，我们在 PCIe Gen5 下剖析了 4.8 万上下文的解码（图 4）：每 token 40.1 ms，其中 GPU1 上传未命中专家 17.4 ms（约 26.5 个、474 MiB），GPU1 内核 13.3 ms，GPU0 内核 4.8 ms，两卡都空闲 4.5 ms，几部分基本串行。上传是最大的一块。

4.2 解码注意力只取 top-k 行

剖析还显示 GPU1 内核里有约 3 ms 花在 `concat_cont` 上，而且随上下文增长：每层每个 token 都把滑动窗口和整个压缩 KV 拼成一个张量（4.8 万上下文时每次约 40 μs），再由 flash attention 按 top-512 掩码挑行。单个查询时这是浪费的：它只会看到 512 个压缩行。我们改为先用 `get_rows` 取出这 512 行和它们的掩码，再和滑动窗口拼接，补齐到 256 的倍数后走普通 flash attention。

4.8 万上下文的解码从 27.7 提升到 30.6 tok/s，12.8 万上下文从 23.9 提升到 30.9 tok/s（都是同一提示词第二次生成，§4.4），解码速度基本不再随上下文变长而下降。server 上 temperature 0 生成的 top-20 KL 约 0.003，高于解码器层那一档底线。我们用 float64 对拍确认了原因：给定相同输入时，两条路径看到的是同一组 640 行，和 float64 的误差都约 5e-4

（F16 舍入）；第 20 层的差别来自上游——第 2–19 层约 5e-4 的舍入差别经 MoE 在分数接近的专家之间选择而被放大，这正是「经过编码器层」那一档的机制。

4.3 按热度准入的未命中分流

原设计是把未命中全部交给 CPU 计算，只按静态热表换入。剖析后发现这行不通：LRU 的高命中率靠的是每个未命中都换入，不换入时命中率会掉到静态热表的水平（每 token 约 47 次未命中），CPU 计算反而比上传还慢；CPU 算完再把同一个专家上传，又要从内存读两遍。

我们改为按热度准入。每层由一个 CPU 算子规划：命中的照常读缓存；未命中的按衰减热度（每层每步乘 0.99）挑出一定比例，立即异步上传进 LRU 槽位；其余由 CPU 用主机内存里的原始专家计算，GPU 那边读 bank 末尾的全零槽位，down 之后两边相加。计算图里 CPU 分支先于 GPU 分支展开，规划算子在 CPU split 开头发出上传，CPU 计算其余未命中时 DMA 正在进行，每个专家只从内存读一遍。

表 7 是上传比例的扫描。只上传一半，命中率不降：按热度挑出来的才进缓存，冷门专家不会把缓存冲掉。最终取 0.5（10-28 专家缓存加到 20.5 GB 后重扫，改为 0.3，§4.5）：4.8 万上下文 36.1 tok/s，12.8 万上下文 36.2 tok/s，KL 约 2e-4，低于解码器层底线。这两个数和表里的速度一样，都是同一提示词第二次生成测出的，偏高；实际会话见下节。

表 7 | 未命中上传比例的扫描（4.8 万上下文，已开启 top-k 收集，复用提示词缓存后生成 256 token，即同一提示词第二次、一字不差的生成）。

上传比例	解码（tok/s）	命中率	对全部上传的 top-20 KL
全部上传（原缓存）	30.6	75.5%	—
1.0（新路径）	30.2	75.5%	0（逐位一致）
0.5	36.1	75.9%	2.7e-4
0.4	35.7	75.1%	2.2e-4
0.3	36.4	74.5%	1.7e-4
0.2	36.0	73.1%	2.2e-4
0（全部交给 CPU）	16.1	0%	1.8e-4

4.4 实际会话中的解码速度

Codex 的实际会话（上下文约 10 万 token）解码只有 30–31 tok/s，明显低于表 7 的 36。差别在测法：表里的速度来自同一提示词的第二次生成，temperature 0 下它和第一次一字不差，第一次用过的专家正好还在 LRU 缓存里，命中率偏高；同一脚本里第一次生成只有 32–35 tok/s。Codex 自带的测速在短上下文下得到 47–49 tok/s，也是同一段生成重复跑出来的。

我们改用只追加的会话测量（`decode_modes.sh`）：首轮放文档和工具定义，之后每轮追加上一轮的回答和一个新问题，生成 256 个 token，轮换贪心或默认采样、流式或不流式。9.6 万上下文 27–31 tok/s，4.8 万上下文 27–29 tok/s，四种方式没有差别，和 Codex 的实际会话一致。所以解码既不受流式和采样影响，上下文从 4.8 万到 9.6 万也不变慢。

表 8 是剖析结果。和上下文长度有关的内核（注意力、索引器打分、top-k、取行和拼接）在

表 8 | 只追加的多轮会话中每 token 的时间（nsys，内存 3600 MT/s）。

部分	9.6 万上下文	4.8 万上下文
GPU1 上传未命中专家（约 20 个、360 MiB）	14.5 ms	14.7 ms
GPU1 内核	10.5 ms	10.4 ms
GPU0 内核	4.2 ms	4.2 ms
两卡都空闲	7.9 ms	8.1 ms
合计（nsys 下）	37.1 ms	37.5 ms

9.6 万上下文时合计只有约 2 ms，top-k 已经是基数选择。一个解码器层的时间线是：GPU1 先算上一层专家的合并、本层注意力和路由（127 个内核，约 0.47 ms）；路由结果和本层输入传回主机后，上传 1 个未命中的专家（3 × 6.3 MB，约 0.70 ms），同时 CPU 算另一个（约 0.79 ms）；之后 CPU 结果传回、启动下一段计算图（约 0.09 ms）。上传和 CPU 计算同时从内存读约 37.6 MB，用了约 0.8 ms，已经到内存带宽的上限；而 GPU 上能挪到上传期间去算的只有本层命中的专家，每层不到 0.1 ms。所以解码剩下的时间主要是每层串行的未命中处理，把上传和命中专家的计算并行起来，每 token 最多省 2–3 ms。

内存提到 5200 MT/s 后（第 7 节），Codex 又用每次不同的短提示词测了一次：生成 128 或 256 个 token，9 次的中位数是 40.3 tok/s（36.8–44.8）。同一时段 Codex 自己的会话（上下文约 10.4 万）是 32.4 tok/s，和表 13 的 31–33 一致。短上下文更快，一部分是注意力相关的开销更少（9.6 万上下文时约 2 ms/token），一部分是生成内容不同、专家缓存命中率不同；到 4.8 万以上，解码就基本不再随上下文变慢。

接入 Hermes 后，长会话（6–9 万上下文，代码和工具输出）里有一半请求的解码掉到 14–26 tok/s。为此我们加了解码统计（`0016`）：每 100 个解码 token 在日志里打印一行，含每 token 耗时、专家缓存命中率、每 token 命中、上传和 CPU 计算的专家数，以及磁盘缺页次数。慢的窗口里命中率、上传数和缺页都和平时一样，也排除了 CPU 被别的进程抢、降频、PCIe 链路代数和语法采样；唯一的相关是忙着的 CPU 核更少，即 OpenMP 线程在层与层之间睡下、再被唤醒。设 `GOMP_SPINCOUNT=100000000` 让线程多空转一会儿，平均从 43.8 降到 35–36 ms/token，超过 50 ms 的窗口从 29% 降到 7–19%。剩下的慢窗口在 10-27 关机、GPU1 出过一次开机故障（§6.3）并重启之后没有再出现：之后 547 个窗口的中位数是 31.1 ms，最慢的 10% 为 32.8 ms，没有超过 50 ms 的。它们和 GPU1 当时的状态有没有关系，我们没有直接证据。

4.5 腾出显存给专家缓存

解码的速度几乎只取决于专家缓存的命中率。按几次测量拟合，每 token 约 15.9 ms 是固定的，每个未命中的专家再加约 0.45 ms（每 token 在第 22–39 层共挑 108 次）；按这个关系推算，不开缓存约 15.5 tok/s，和最早的实测相符。19.2 万上下文时缓存只有 12 GB（§8），10-28 我们从三处找回显存（表 9）。

计算图的两处浪费（`0019`）。我们给 ggml 的分配器加了 `GGML_ALLOC_TRACE`，记下每次分配计算图时每个张量的分配和释放。`0014` 每层的全零张量是计算图的叶子，图开始前就全部分

配（GPU0 上 18 × 240 MiB），改为用到时才生成后，GPU0 加载后从 94,279 降到 91,694 MiB。CED 下没有查询行的层照样算查询，结果没人读，却一直占着显存（GPU1 上 17 × 256 MiB）；去掉后预填充还快了 3–5%（9.6 万 token 从 42.6 降到 41.2 s），输出不变。

索引器按查询分块（`0020`）。GPU1 计算缓冲区的大头是预填充时稀疏索引器的打分表和注意力掩码，大小正比于「ubatch × 上下文」。照 DeepInfer 的做法让打分和注意力每次只算一段查询后，256 个查询一段时计算缓冲区从 6.9 GB 降到 3.2 GB，大段预填充慢 4–8%，对不分块 KLD 0.0079，在编码器那一档底线以内；解码的计算图和不分块完全相同。第一版分块反而更占显存：每层每段各建一个掩码视图，调度器给每个视图都往 GPU 拷一份，同一次切分的输入又在切分开始时全部分配，改为每个计算图共用一组视图才降下来。分块大小扫过 256、512、1024：GPU1 峰值 95,880 / 95,906 / 96,684 MiB，9.6 万 token 提示词 44.0 / 42.8 / 42.0 s；1024 的余量只剩约 1.2 GB，生产用 512。

GPU0 的空余显存。缓存只能放在用它的层所在的卡上：第 22–39 层都在 GPU1，GPU0 上空着的约 6 GB 用不上。我们用 `-ot` 把第 12 层的 gate、up 专家张量（各约 2.3 GB）放到 GPU0，GPU1 腾出的空间加给缓存。第 12 层因此每次多跨卡两次，预填充慢 1–2%；只挪 down 一个张量、缓存 18 GB 时，解码只快 2.9%。

表 9 | 逐步加大专家缓存（2026-10-28，9.6 万上下文、只追加的多轮会话，每档 12 轮）。每一行是同一次测试里改动前后的对比，所以相邻两行的数接不上。

改动	槽位	解码（tok/s）	命中率
索引器分块，缓存 12 → 16 GB	670 → 890	31.95 → 35.42	68.4% → 74.7%
第 12 层 gate、up 放到 GPU0，16 → 20.5 GB	890 → 1,140	35.54 → 38.06	74.0% → 79.0%
上传比例 0.5 → 0.3	1,140	37.75 → 38.64	77.3% → 77.2%

两步加大缓存后，命中率从 68% 升到 79%，解码快了约 19%，和拟合的预测（20.5 GB 时 38.3 tok/s）一致。显存几乎用满：GPU0 峰值 96,386 MiB，GPU1 约 95,900 MiB，余量约 1.5 和 2.0 GiB。9.6 万 token 提示词的输出在三种缓存配置下逐 token 相同；19.3 万 token 时 top-20 KL 约 4e-4，因为缓存大小不同，交给 CPU 算的专家也不同。

缓存变大后，我们又扫了一遍未命中的上传比例（`admit-sweep.sh`）：0.1 / 0.2 / 0.3 / 0.5 时解码 36.44 / 37.83 / 38.64 / 37.75 tok/s，命中率 72.8 / 75.5 / 77.2 / 77.3%。比例太低时缓存补不上新用到的专家；0.3 的命中率已和 0.5 一样，每 token 却少传约 5 个专家。5200 MT/s 下那次扫描（第 7 节）也是 0.3 略好，生产改为 0.3。上线后（10-28 晚，Hermes 和 Codex 的 53 个请求，上下文 4.6 万到 14.9 万），解码统计的 660 个窗口平均每 token 27.2 ms（36.8 tok/s），最慢的 10% 为 29 ms，没有超过 50 ms 的，命中率约 79%，和测速一致。

4.6 剩下的时间

缓存加到 20.5 GB 后（上传比例还是 0.5），我们用生产设置重新剖析了 9.6 万上下文的解码（`profile-prod.sh`，图 4 第一行）。nsys 下每 token 30.4 ms：GPU0 计算 4.3 ms，GPU1 上传未命中的专家 8.0 ms（约 11.5 个、207 MiB，28.6 GB/s），GPU1 计算 10.4 ms，两卡同时空闲 7.7 ms；四段完全串行，GPU1 的上传和计算没有重叠。每 token 有 229 次 `cudaStreamSynchronize`。

`nsys_layer.py` 按前后事件给两卡同时空闲分类：约 2.2 ms 是 CUDA graph 的启动（每 token 约 25 次、每次约 90 μs），约 3.8 ms 是没有上传的层里 GPU 干等 CPU 算专家。一层的时间线里，上传挡在这一层的专家计算之前，下一层又要等这一层的结果；能和上传同时做的只有本层命中的专家，边传边算估计每 token 只能省 1–2 ms。

几个便宜的开关也试过（`quick-knobs.sh`、`threads-sweep.sh`，9.6 万上下文、每档 12 轮）。关掉 CUDA graph，解码从 38.99 降到 36.77 tok/s（−5.7%）；开 ggml 的图优化是 39.20 tok/s，在误差以内；都不用。CPU 线程 12 / 16 / 24 / 32 是 38.54 / 37.96 / 37.87 / 36.89 tok/s，差别在噪声以内（命中率本身有约 2 个百分点的起伏），保持 16：没有上传的层里 GPU 等的是 CPU 读专家，每个约 18 MB，16 个线程已基本占满内存带宽。每个未命中的专家从上传改给 CPU 算只省约 0.12 ms（上传约 0.66 ms，CPU 算也在关键路径上），所以把上传全挪到后台，估计也只能再快 5–10%。

4.7 测过但没有采用的两条路

每 token 少用一个专家。 Strata [4] 在四张 R9700 上默认的「Faster」模式每 token 只用 5 个专家。我们用 `--override-kv deepseek41.expert_used_count=int:5` 试了（`experts5.sh`），选中专家的权重照常归一化。4.8 万上下文、只追加的会话里，解码从 32.6 提到 36.8 tok/s（+13%）：丢掉的是排第 6、较冷门的专家，未命中少了 26%，命中率从 65.8% 升到 69.5%；每轮的小预填充也快约 10%。但对 6 个专家的 KLD 是 0.032、top-1 一致率 91.9%，PPL 从 5.863 升到 5.954（+1.6% ± 0.35%）：是数值底线的 3–6 倍，约为 3.0 bpw 量化（KL 0.0545）的六成，抵掉了换 1017 的 PPL 改进的一半。Strata 在 2.25 bpw 上测不出差别，是因为它的量化误差（KL 0.107）盖住了。生产仍用 6 个。

DSpark 投机解码。 fork 的 `draft-dspark` 支持 V4 风格的草稿模型，1017 的侧车（§9）可以直接用（`dspark.sh`）。6.4 万上下文、贪心写代码时，解码从 31.1 提到 36.2 tok/s（+16%），每次验证平均接受 4.3–4.5 个 token，草稿接受率 66–70%。收益的上限很低：一次验证要主模型算 6 个 token，用到的专家更多，未命中成倍增加，约 120 ms，是单 token 的 3.8 倍，5 个草稿全部接受也只快约 1.6 倍。另有三个问题。草稿模型的权重 7.6 GB、计算缓冲区 2.3 GB，专家缓存要降到 6 GB 才放得下，这本身就让 4.8 万上下文的解码从 32.6 降到 28.0 tok/s，抵消了收益。草稿模型要读第 37–39 层的隐状态，预填充时 CED 跳不过解码器层，4.8 万 token 的首轮预填充从 23.4 s 变成 91.7 s。长提示词还让草稿模型的 KV 位置断档（写到 33,112 后，下一批从 48,222 开始），请求以 HTTP 500 失败。贪心输出和不开投机时在第 104–315 个字符处开始分岔，分岔处是近似等价的选择，来自批量验证和单 token 解码的数值差别。暂不采用。

5 预填充

原版对提示词也跑完 40 层：每个 2,048 token 的 ubatch 都要把主机内存里 121 GiB 的解码器专家读一遍。4.8 万 token 的提示词因此要 135 s（约 355 tok/s）。

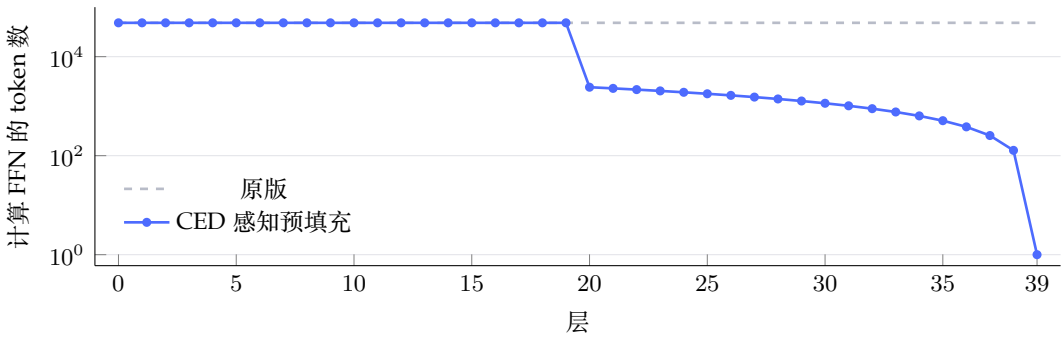


图 5 | 4.8 万 token 提示词每层计算 FFN 的 token 数（按实现中的规划规则计算，没开 DSpark 时不留余量）。编码器层全部计算；解码器层只计算后面还能通过滑动窗口看到的 token。

5.1 CED 感知预填充

官方参考实现对提示词也跑满 40 层，CED 体现在 KV 的共享方式上：解码器层只有滑动窗口用到各层自己的隐状态。因此可以精确地裁剪：第 0–19 层全部 token 都算；第 39 层只算最后一个 token；第 L 层（20–38）只需算最后 $128 + 127 \times (38 - L)$ 个 token——每往前一层，滑动窗口让依赖往前延伸 127 个 token。实现原本在末尾多留 256 个位置，给投机解码的回退用；没开 DSpark 时 DSV4 不能部分回退，后来改为默认不留（§5.5）。这样解码器层合计约 2.4 万个「token × 层」，与提示词长度无关；4.8 万 token 的提示词在原版中是 $20 \times 48{,}240 \approx 96$ 万个（图 5）。

实现分两步。第一步（`0003`）只跳过 FFN，注意力仍对所有 token 计算，KV 写入完全不变；被跳过的 token 之后各层的隐状态不正确，但只会进入没有后续 token 读取的滑动窗口。每次 `llama_decode` 按自己的末尾规划，所以 server 用 `-b 65536` 让整段提示词进一次调用。4.8 万 token 的预填充从 135 s 降到 41.8 s，8k/16k 上下文对拍 KLD 0.00006 / 0.00004，top-1 一致率 99.8%。

第二步（`0005`）先剖析：此时 41.8 s 里第 20–39 层的注意力占 19.9 s。我们让这些层仍为所有 token 写 KV，但只对 CED 子集取出查询和掩码行计算注意力。预填充降到 27.9 s（1,715 tok/s），KLD 0.00004–0.00007，temperature 0 的输出和第一步一字不差。

5.2 Engram 预取

Engram 表以 mmap 映射且设了随机访问提示，每次缺页都同步读一个 4 KiB 页，2,048 个 token 约需 2.4 s，瓶颈在缺页延迟而不在 SSD 带宽。由于行号只取决于输入 token，我们在 `llama_decode` 拿到整批后立即算出全部行号，由 8 个后台线程按地址排序后用 `MADV_WILLNEED` 预读。页缓存冷时，4.8 万 token 的预填充从 91.1 s 降到 54.3 s，冷读开销从约 39 s 降到约 2 s；热缓存时不变，结果与不预取完全一致。冷读已基本藏在计算后面，io_uring 和按行缓存等后续步骤暂缓。

5.3 补上官方稀疏索引器

fork 原本对 V4.1 的压缩 KV 做稠密注意力。压缩行在 512 以内时与官方的 top-512 选择等价，更长时就不同了：压缩比 1 的层超过约 512 个 token、压缩比 2 的层超过约 1,024 个 token 之

后。智能体的上下文几乎都在这个范围里，所以这首先是正确性问题。

我们按官方 `inference/model.py` 补上了完整的索引器：KV 源层写 FP4 索引键（每 32 个一块，缩放 $2^{\lceil \log_2(\text{amax}/6) \rceil}$），索引源层计算 $\sum_h \text{ReLU}(q_h \cdot k)\, w_h$ 并选 top-512，第 20 层挑候选块，其余层沿用最近索引源的结果；flash attention 只看选中的位置。512 token 以内与稠密版 KLD 为 0；8k/16k/32k 上下文时，稀疏版的 PPL 比稠密版低 0.9% / 3.1% / 0.23%（表 10）。由于注意力只看 640 行，4.8 万 token 的预填充也从 27.9 s 降到 22.7 s，12.8 万 token 为 51.8 s（2,464 tok/s）。

表 10 | 稀疏索引器对稠密注意力（perplexity，每档 4 段）。

上下文	PPL（稠密）	PPL（稀疏）	变化	稀疏对稠密 KLD
8k	6.023	5.972	−0.9%	0.057
16k	3.009	2.916	−3.1%	0.094
32k	1.0027	1.0004	−0.23%	0.012

为确认实现与官方一致，我们导出第 2、20、24 层的索引器输入输出，用三种算术重算 top-512 并比较重合率（表 11）：全程 float32；fork 的算术（Q8_0 矩阵乘把激活量化成 int8）；官方的算术（bf16 激活，量化成 e4m3 后做 FP8 GEMM）。fork 与自己的算术重合 99.98% 以上，说明 RoPE/YaRN、FP4、权重缩放、可见性和 top-k 每一步都对；fork 离 float32 的距离（98.7–99.2%）比官方算法自己离 float32 的距离（97.1–98.7%）还近。索引键 99.99% 以上逐位一致，不一致的都是恰好落在 FP4 中点上的单个元素。

表 11 | top-512 集合的平均重合率（4,027 token 提示词）。

层	fork 对 int8 参照	fork 对 float32	官方算术对 float32	fork 对官方算术
2	99.99%	99.16%	98.67%	98.63%
20	99.98%	98.67%	97.14%	97.11%
24	99.98%	98.85%	97.40%	97.36%

5.4 大批次分流

多轮会话里每轮新增的 token 不多，但只要批次超过 32 个 token，第 22–39 层几乎全部 384 个专家都会被用到，调度器把这些大批次的专家矩阵乘卸载到 GPU0，每层整批上传。一轮 1,610 token 的预填充（7.0 s）中，GPU0 上传 145 GiB 用了 5.4 s，而 CPU、GPU1 的链路和专家缓存都闲着。这部分的下限是把这些专家从内存读一遍，约 3 s。

我们把未命中分流的思路推广到大批次：每层一个 CPU 算子统计每个专家的 token 数，按从少到多把专家交给 CPU，直到 CPU 的预计时间追上其余专家的上传时间；其余照常上传到 GPU0。为此给 CUDA 的 MMQ 补上了 id = −1 的处理（跳过并把输出清零），上传回调直接用规划算子在主机上的结果，不必从 GPU 读回 id。

第一版的 CPU 分支处理整块张量（全部 token × 6 个专家），而 CPU 实际只算约 5% 的行，量化、SwiGLU、清零和 252 MB 输出的回传把收益全吃掉了，比不分流还慢。改为紧

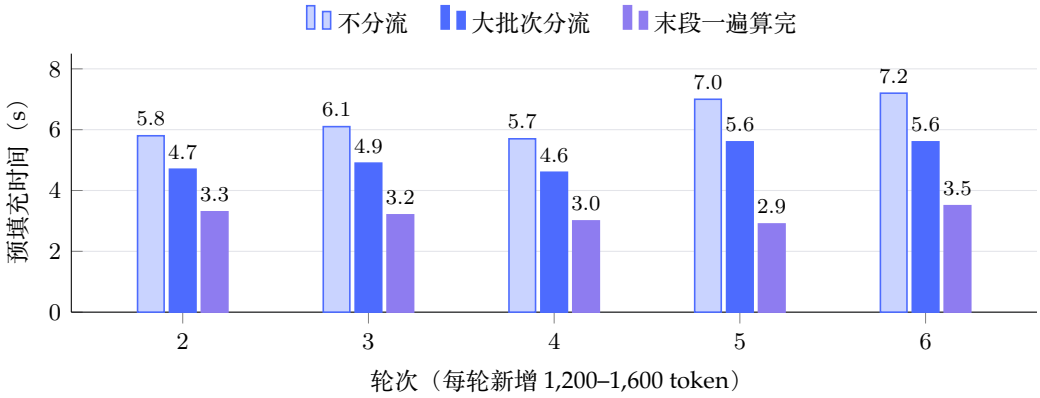


图 6 | 多轮智能体会话中每轮的预填充时间（首轮为 4.8 万 token 的文档，之后每轮回填工具结果或追问；内存 3600 MT/s）。末段一遍算完见 §5.5。

凑批次后——只取出交给 CPU 的「token × 专家」（最多 1,024 行），在 GPU1 上取出这些 token，CPU 上按一行一个专家计算，`set_rows` 写回后与 GPU0 的结果相加——2,048 token 一批的预填充从 4.88 s 降到 4.24 s，KLD 0.00046。多轮会话之后每轮的预填充从 5.7–7.2 s 降到 4.6–5.6 s，首轮从 30.6 s 降到 27.9 s（图 6）。

这一步离下限还有约 30 ms/层：GPU0 的上传暂存区在 gate、up、down 三份权重之间复用，每段之间都要同步，CPU 要等 gate 那批传完才能开始；CPU 计算和 DMA 又共享 44 GB/s 的内存带宽。合并的几步要固定在 GPU0，否则 GPU1 的计算缓冲区会大 3.7 GB，12.8 万上下文时放不下。实现中还发现并修复了 fork 的一处问题（`0013`）：`ggml_backend_sched_reserve_size` 在切分计算图之前重置调度器，清掉了构建计算图时手动指定的后端，按错误的切分估算计算缓冲区。

5.5 末段一遍算完

CED 让每次 `llama_decode` 调用都按自己的末尾跑解码器层，凡是超过 32 个 token 的调用都要把第 22–39 层几乎全部专家从内存读一遍。server 为了建检查点，会在最后一条用户消息开头、末尾前 4 + n_ubatch 和末尾前 4 处拆调用。生产日志里 Codex 的大多数轮次因此被拆成两次大调用（例如 1,772 + 866 + 4），比同样长度只有一次调用的轮次多花 1.5–2 s；一次调用里末段跨两个 ubatch 时，也要读两遍。我们做了三处改动（`0015`）：

- 提示词最后 2,700 个 token 里不再按用户消息开头和末尾前 4 + n_ubatch 拆调用，只留末尾前 4 个 token 那次，下一轮靠它从助手前缀处接着算。会话的第一个检查点照旧，Codex 的辅助请求靠它复用系统提示。代价是这两类检查点没了：只有改写最后一条用户消息时才用得上，这时 server 退回上一轮的检查点，多算这一轮。
- 一次调用的余数放进第一个 ubatch，最后一个 ubatch 是满的 2,048 个。原来余数在最后，最后一个 ubatch 常常只有几百个 token，第 20–32 层左右都要在前一个 ubatch 里再算一遍；现在前一个 ubatch 只剩第 20–22 层的少量 token。
- 末尾的余量默认等于回退环的深度 `n_rs_seq`，没开 DSpark 时为 0。我们按规划公式逐个位置核对过，余量为 0 时下一个 token 需要的滑动窗口状态仍然完整。

表 12 | 末段一遍算完的效果（`agent-turns.sh`，内存 3600 MT/s）。

配置	首轮 4.8 万 token	之后 5 轮（1,200–1,600 token）
改动前	27.9 s	4.6–5.6 s
末段不拆调用	25.3 s	3.2–3.7 s
+ 最后一个 ubatch 对齐到满	24.4 s	3.2–3.7 s
+ 余量为 0	24.0 s	2.9–3.5 s

新提示词按外部测速的方式（关闭 prompt 缓存，每档 3 次取中位数）测，2,324 token 从 5.57 s 降到 3.69 s，4,277 token 从 8.80 s 降到 4.34 s。对拍方面，最后一个 ubatch 对齐对改动前的 KLD 是 0.0092、top-1 一致率 95.4%，和只把 ubatch 从 2048 改成 1024 的 0.0090、96.0% 一样；余量 0 对余量 256 生成的 256 个 token 全部相同（top-20 KL 4.6e-4）。我们还用 token 数组直接构造提示词，让调用长度除以 2,048 余 0、1、2、7–9、31–33、127–129、1000、2047，各测约 2 千、4 千、2 万 token，42 个请求全部正常。

5.6 大批次用缓存

末段一遍算完之后，每轮的预填充还有一大块固定开销：生产日志里 10-28 下午的 92 次预填充，拟合出每次约 1.84 s，加每个新 token 约 0.98 ms，Hermes 一轮只新增 505 个 token 也要 2.2 s。原以为 CED 让解码器层每轮只算末段约 128 个 token。我们用 `0022` 打印每个大批次用到的专家数和其中缓存已有的（`big-coverage.sh`），实测每轮新增 1,100–2,800 token 时，第 22–38 层每层每批约 860–1,140 个 token、用到约 240 个专家：CED 下解码器层仍要算大部分新 token。这些专家按个数只有 22% 在 GPU1 的缓存里，按挑选次数却是 55%：缓存里是解码常用的专家，分到的 token 也多。

当时的规划每批把约 126 个专家交给 CPU（其中 13 个在缓存里），116 个从内存上传到 GPU0 算，其中 40 个其实已在 GPU1 的缓存里：缓存只能复制给它所在的卡，大批次却在 GPU0 上算，只好照样从内存上传。剖析一轮（新增 1,609 token，3.2 s）：GPU0 从主机上传专家 37 GiB、用了 1.37 s，GPU0 计算 0.52 s，GPU1 计算 0.61 s（注意力 0.22 s，其中索引器 0.13 s），GPU1 的传输 0.11 s，两卡同时空闲 0.63 s，也基本串行。

`0023` 让规划记下缓存已有专家的槽位，这些专家直接在 GPU1 上用槽位算（gate、up、激活、down，再加权求和），其余的照 `0014` 分给 CPU 和上传到 GPU0。计算图里先发 GPU0 那路，再发 GPU1 这路，最后 CPU 算，三边同时进行，结果相加。同一个会话关、开各跑一遍（`big-cache.sh`，图 7），同样长度的轮次快 0.3–0.4 s（约 11%），整场约 20% 的大批次专家改从缓存算；长的新提示词基本不变，因为缓存是冷的，只命中 16%。GPU1 加载后的显存和峰值都不变，新路径的中间张量和峰值错开了。对拍时先用短提示词贪心解码 512 个 token，把两次的缓存填成一样，再跑 9.6 万 token 的提示词：32 个输出 token 相同，top-20 KL 1.7e-4。

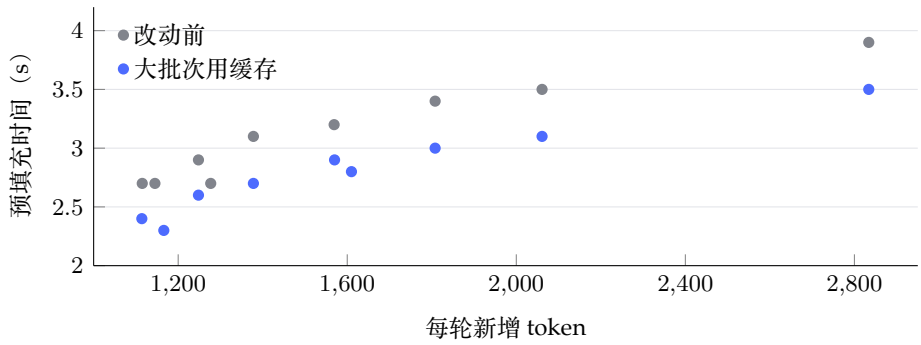


图 7 | 多轮智能体会话中每轮的预填充时间（`big-cache.sh`，首轮 9.6 万 token，之后 9 轮；内存 5200 MT/s、专家缓存 20.5 GB）。两次会话的生成不完全相同，有两轮的新增 token 数不一样。

6 服务与智能体负载

6.1 多轮会话的缓存复用

V4.1 的缓存不能回退到滑动窗口中间，只能退回检查点。server 在每条用户消息的开头、最后一条用户消息开头和提示词末尾附近创建检查点。模拟会话中，从第 3 轮起每轮只重算上一轮生成的回复加 4 个 token：回复经模板重新渲染后与生成时的 token 不完全一样，server 退回到「Assistant」前缀处的检查点。

6.2 prompt 缓存的选择

在 Codex 的 6 个真实请求中（预填充合计 76 s，生成 129 s），我们发现 server 只有一个槽位，Codex 的辅助请求（与主对话共享约 4,500 token 的系统提示和工具定义，只问一句）会把主对话挤出槽位。server 已把主对话存进内存里的 prompt 缓存，但主对话回来时，槽位里的辅助请求与它仍有 94% 的前缀相同，而 server 只在保留比例低于一半时才去缓存里找，并且按保留比例挑选，长的条目反而落选。

我们改为按「能直接用上的 token 数」比较槽位和缓存条目——对检查点受限的缓存，就是不晚于分叉点的最近检查点的位置——多出 256 个以上就换入；只用到一小部分的条目复制一份，原条目留在缓存里。另一个问题是辅助请求自己也从头算：系统提示末尾有检查点，但 server 要求检查点比分叉点早至少 128 个 token。这是为保存滑动窗口的检查点定的规则；V4.1 的检查点是那个位置的完整状态（`pos_min == pos_max`），在分叉点之前就能接着算。修复后，同一场景里辅助请求从 8.4 s 降到 0.3 s（只重算 13 个 token），主对话的下一轮从内存缓存换回，只重算 216 个 token、2.2 s。

6.3 真实负载暴露的缺陷

- **未分配的输入张量。**CED 让某些层在某个 ubatch 里没有 token，依赖它们的计算被整段丢掉，预先建好的输入张量就不会分配，往里写数据时断言失败。这在 CED 行映射（提示词约 2,500–16,000 token）和候选块输入（约 1.7 万 token 以上、长度除以 2,048 余约 114–622）上各出现一次，都只在特定长度下触发，测速用的 4.8 万和 12.8 万 token 恰好都没碰到，由 Codex 的请求暴露。

- **全零槽位重复。**未命中分流最初只有一个全零槽位。9–32 token 的批次里，同一 token 的多个 CPU 未命中指向同一个槽位，CUDA MMQ 的重排每个专家每个 token 只记一行，漏掉的行号未初始化，量化内核越界。改为每个 token 的第 k 个专家用第 k 个全零槽位。

- **GPU1 开机故障。**四天里 GPU1 在开机时出了四次问题，GPU0 一次也没有：10-24 链路只训练到 Gen4，主机到 GPU1 只有 14.4 GB/s，未命中上传慢了一倍，解码从约 25 掉到 17 tok/s（图 4）；10-25 改 BIOS 后第一次开机只到 Gen3，只有 1.78 GB/s；10-27 关机后再开机，驱动初始化不了 GPU1 的 GSP 固件（`RmInitAdapter failed`、`WPR2 already up`），重启后又只到 Gen4（14.47 GB/s），再重启才正常。测速脚本会先检查链路。10-28 断电时在 BIOS 里把 PCIe 固定为 Gen5（没有重新插卡），之后开机两张卡都是 Gen5 x8；固定之后链路训练不上就不会再降速，而可能整张卡认不到，开机后要先确认 `nvidia-smi` 里有两张卡。

- **测试 server 退出时卡住。**测试 server 收到退出信号时正好接到外部请求，之后 GPU 和 CPU 都是 0%，只能强制结束。生产重启时也碰到过退出清理卡一分钟以上；重启脚本现在等 60 s 后再发一次信号（槽位已先存好，第二次会立即退出），再等 30 s 仍没停就强制结束，然后等驱动把显存还回来再启动，否则紧接着加载会分配显存失败。

- **Codex 的 MCP 工具被丢弃。**Codex 每个请求带的 9 个 `namespace` 类型工具，server 不支持，直接跳过，模型看不到。这个问题还没有修。

6.4 重启后的长对话首次预填充

server 重启后 prompt 缓存是空的，Codex 的下一个请求要把整段对话从头算。2026-10-25 的日志里，一段约 10.4 万 token 的多轮对话用了 85.2 s（1,219 tok/s），而单条消息的 12.8 万 token 提示词只要 51.8 s（2,464 tok/s，§5.3）。差别来自检查点：server 在对话中间每隔至少 8,192 个 token（`--checkpoint-min-step` 的默认值）、在用户消息开头处拆一次调用并建检查点，这段对话因此被拆成至少 16 次调用。每次调用都按自己的末尾跑解码器层，要把第 22–39 层的专家再读一遍；§5.5 的合并只管最后 2,700 个 token。

我们从两头改。一是重启脚本在停 server 之前把槽位 0 连同检查点存盘（`--slot-save-path`），启动后读回：9.6 万 token 的槽位文件约 0.34 GiB，存 86 ms、读 114 ms，重启后的第一个请求接着算，不用重算整段对话。二是把 `--checkpoint-min-step` 调到 32768，没有存档时（例如崩溃后）拆的次数少很多：9.6 万 token、16 条用户消息的对话首次预填充从 77.3 s 降到 52.9 s（`restart-prefill.sh`）。后一项有代价：检查点列表满了以后也按这个间距删减，对话中途分叉时可能要退回很远（§6.7）。

但存档只覆盖槽位 0。主对话、复盘、Codex 的会话轮流用这一个槽位，不在槽位里的只留在内存的 prompt 缓存里，重启就丢了。10-28 因为测试重启了 12 次，其中 3 次之后的第一个请求把整段对话从头算了一遍：15.5 万、12.2 万和 13.9 万 token，分别用了 93、79 和 81 s。例如 21:30 那次，停机时槽位里是一段 5.5 万 token 的对话，重启后来的却是另一段 13.9 万的。下一步是把内存 prompt 缓存里的各段对话也连同检查点存盘，启动后读回。

6.5 对话模板与采样

2026-10-26 接入 Hermes 后，有一次请求的思考越写越长，被客户端以「Repetition Detected」中止，那段思考循环了约 4,700 token。翻看会话记录，思考里「Let me write. / Go.」这样的短句在同一个会话里越来越多：会话的前 2.5 小时一次都没有，之后每条回复里的平均个数从 0.2 涨到 16，最多的一条有 30 个。带工具的对话会保留每一轮的思考，历史里的这类短句一直留在提示词中，模型照着自己之前的写法继续写，越写越多。

我们把 server 用的模板和官方 `encoding/encoding.py` 逐字对照，发现模板沿用了 V4 的写法：开头少了 `<｜System｜>` 和思考强度那一行（`Reasoning Effort: 75 ...`），工具定义和工具结果的位置、拼接方式也不同。按 `encoding.py` 重写后，`template_check.py` 用 10 组对话（system 和 developer 消息、工具定义、多轮工具调用和结果、无工具的多轮思考、几种思考强度）比较 server 的 `/apply-template` 和官方编码，全部逐字一致。对拍中还发现两处：llama.cpp 默认保留历史轮次的思考，官方在没有工具时只保留最后一条用户消息之后的，server 要加 `--no-reasoning-preserve`；llama.cpp 的 Jinja 没有 `from_json`，工具调用的参数要在模板里直接写出。

采样参数原来是 llama.cpp 的默认值（temperature 0.8、top_k 40、min_p 0.05），Hermes 和 Codex 都不在请求里带采样参数，于是改为官方推荐的 temperature 1.0、top_p 0.95，换用 1017 后按其模型卡改为 top_p 1.0（§9）。修复后开的新会话里，这类短句一次也没有再出现；旧会话的历史里已经积了大量短句，要开新会话才能摆脱。

6.6 请求文字里的控制 token

server 原来把整段渲染后的提示词按「识别特殊 token」编码，请求文字里出现的 `<｜User｜>`、`<｜Assistant｜>`、`</think>`、`｜DSML｜` 等字样都会变成真的控制 token，模型会以为对话在正文中间换了轮。我们在生产 server 上确认过；Codex 10-25 有 3 次工具输出含这类字样，智能体读到 llama.cpp 的 `src/llama-vocab.cpp` 时，提示词中间就会多出一个假的结束符。只关掉识别也不够：`<think>`、`</think>`、`｜DSML｜` 在 GGUF 里是用户定义类型的 token，照样会被匹配。

`0018` 按官方 Recipe [8] 的规则编码：只有模板自己写的控制 token 按 ID 编码，其余文字，包括模板里的普通文字，只按基础词表编码。哪些字节来自请求，靠 fork 的 Jinja 引擎给字符串打的输入标记；补丁补上了 `tojson` 和数组 `join` 对标记的传递。这只对 DSV4 的模板生效，续写助手消息的请求走原路径。对拍用 `special_check.py`：§6.5 的 10 组对话，加上 7 组把这类字样放在用户消息、system、工具结果、助手历史、字符串和对象参数、工具定义里的对话，和官方编码器（控制 token 用常量加标记）加去掉 added token 的 HF 分词器逐 token 一致；不含这类字样时和原编码逐 token 相同。40 万字节的提示词编码用 96 ms（原来 104 ms）。输出侧还没有处理：模型自己用普通 token 写出 `</think>` 或工具调用的标签时，解析器仍按文字识别。

6.7 槽位被只共享开头的请求占用

Hermes 每轮结束后会在后台重放整段对话，问模型要不要存技能和记忆（下称复盘）。10-26 晚上一个小时里有三次，主对话的下一条消息要从头重算约 3.6 万 token，每次 20–23 s。原因是三件事叠在一起：复盘请求和主对话只共享前约 4.06 万 token，进来时占了唯一的槽位；server 按共同前缀算保留比例（72%），超过一半就不先把槽位存进内存缓存；而 V4.1 只能退回共同前缀里最新的检查点，当时检查点列表已满，按 `--checkpoint-min-step 32768` 删减后，2 万到 5 万之间一个都不剩，只能退到 20,466。主对话 2 万以后的部分既没有存进缓存，也没有检查点可退。

`0017` 改了两处。一是按检查点算新请求实际能保留多少，比槽位最新的检查点少 2,048 个 token 以上（`ABYSS_SLOT_SAVE_LOST`）就先存进内存缓存，主对话回来时由 §6.2 的逻辑换回。二是把检查点删减的间距和拆分长提示词的步长分开（`ABYSS_CKPT_THIN_STEP=8192`）。`slot-takeover.sh` 复现了这个场景：6.5 万 token 的对话被一个只共享前 3.8 万的请求占用后，下一轮从重算 26,438 个 token（15.0 s）变成 30 个（0.5 s），32 个输出 token 和改动前相同。生产上第一次触发是 Codex 的一个辅助请求，主对话随后只算了 656 个 token。删减间距那一项在这个测试里没有被触发（36 轮还没有填满检查点列表），还要补测。

6.8 智能体客户端的额外请求

除了主对话，Hermes 还会发几类请求，都和主对话抢唯一的槽位：

- **复盘**：见上一节。它可能生成上千 token（一次写技能文件生成了 5,700 token），用户的新消息会让它停下。
- **上下文压缩**：窗口用到约 75% 时，先让模型写摘要，再把摘要和最近的消息从头预填充。有一次摘要生成了 8,189 token、用了 259 s，接着重新预填充 36,711 token、22.7 s。我们让压缩请求不思考（Hermes 的 `auxiliary.compression.reasoning_effort: none`）。另有一次，用户在压缩进行中发来消息，压缩被取消，但请求没有断开，server 仍把 8,490 token 的摘要生成完，用户的消息多等了约 3.5 分钟。
- **命令审批**：默认的 smart 模式让模型审批命中危险规则的终端命令，一天约 23 次，每次的提示词约 800 token、从头算，4.4–4.7 s。改为由用户在 Telegram 上批准（manual）后，这类请求就没有了。

7 把内存提到 5200 MT/s

4 × 48 GB 双 rank、每通道两条时，AMD 默认只跑 3600 MT/s。内存条标称 5600，SPD 里的 XMP 和 EXPO 配置都是 DDR5-6000、1.40 V。我们在 BIOS 里设 EXPO I，再把频率降到 DDR5-5200，并关掉 Memory Context Restore。stressapptest 用 150 GB 压 1 小时通过，没有硬件错误，内存条最高 62 °C；同一编译重跑的对拍和改之前一致，差别和这个编译两次运行之间一样。

各项快了 6–12%，少于我们预计的 15–20%。解码时上传到 GPU1 受 PCIe 限制，每个专家约 0.7 ms，和内存频率无关，只有 CPU 算的那一半变快了。内存带宽也只提高了 23%，理

表 13 | 内存从 3600 提到 5200 MT/s 前后（2026-10-25）。

测试	3600 MT/s	5200 MT/s
内存纯读	44 GB/s	54 GB/s
两卡同时从主机读	43.4 GB/s	50.6 GB/s
单卡从主机读	28.9 GB/s	28.9 GB/s
9.6 万上下文多轮会话解码	29.0–31.1 tok/s	30.7–33.0 tok/s
多轮智能体首轮 / 之后每轮	24.0 / 2.9–3.5 s	23.1 / 2.7–3.2 s
新提示词 604 / 2,324 / 4,277 token	2.59 / 3.69 / 4.34 s	2.27 / 3.35 / 4.01 s

论上是 44%，可能是内存控制器频率被自动设成了内存频率的一半，还没有确认。专家缓存的上传比例（0.5）是在 3600 MT/s 下定的，我们在 5200 MT/s 下重扫了一遍（4.8 万上下文、只追加的会话、每档 6 轮）：0.3、0.4、0.5、0.6 分别是 33.8、33.6、33.3、31.2 tok/s，0.3–0.5 的差别在轮间波动以内，保持 0.5。10-28 专家缓存加到 20.5 GB 后再扫，0.3 仍略好，生产改为 0.3（§4.5）。

8 把上下文扩到 19.2 万

智能体会话很容易逼近 12.8 万的上下文。Hermes 在窗口用到约 75% 时压缩上下文：先让模型写一份摘要，再把摘要和最近的消息从头预填充。2026-10-26 的一次压缩用了约 4.7 分钟，其中摘要 8,189 token、259 s，重新预填充 36,711 token、22.7 s。它每轮结束后的后台复盘要重放整段对话，上下文已经长到约 11.8 万 token。Codex 则在 11 万 token 时自动压缩。

显存的限制不在 KV。V4.1 的 KV 是压缩过的，从槽位存档估算每 token 约 5 KB（88,630 token 连同 29 个检查点共 443 MB），25.6 万上下文也只要约 1.2 GB。随上下文变大的是 GPU1 的计算缓冲区，主要是稀疏索引器给压缩位置打分的临时空间，大致和「ubatch × 上下文」成正比：12.8 万上下文时 server 启动预留 4.5 GiB，跑长提示词时涨到 6.1 GiB，GPU1 峰值 96,686 / 97,887 MiB。

我们用 `ctx-size.sh` 逐个试了几种配置（表 14）。每种配置记下加载后的显存，测试全程每 0.5 s 采样一次取峰值；先跑一段 9.6 万 token 的提示词，贪心生成 32 个 token，用来在配置之间对拍；再在开头放一句暗号，灌满语料直到接近上限，最后问暗号，看能否找回，同时测满上下文时的预填充和解码。

有两种腾显存的办法。一是把 ubatch 减半，计算缓冲区约减半，25.6 万上下文的 GPU1 峰值也只有 94,966 MiB；代价是大段预填充慢约 24%（9.6 万 token 从 43.2 s 到 53.4 s），而且智能体每轮只新增 1–3K token，在 ubatch 2048 下多是一个 ubatch，减半后要分成两个，第 22–39 层的专家要从内存多读一遍，这部分没有实测。二是保持 ubatch 2048、缩小专家缓存：12 GB 时 19.2 万上下文的 GPU1 峰值 95,604 MiB，还剩约 2.3 GB，比原配置的余量还大，因为 19.2 万时启动预留的计算缓冲区已经够用，跑长提示词时不再上涨。我们采用第二种；10-28 修掉计算图的浪费、让索引器分块后，专家缓存又加回 16 GB，再借用 GPU0 的空余显存加到 20.5 GB（§4.5）。Hermes 的压缩阈值随之从约 9.8 万推迟到约 14.7 万，Codex 的自动压缩从 11 万调到 16.5 万。

表 14 | 各上下文配置的显存与速度（2026-10-27，内存 5200 MT/s）。解码是灌满上下文后生成 256 个 token，开了 `ignore_eos`，答完暗号后的内容容易重复、命中偏高，只适合互相比较。

上下文、ubatch、专家缓存	GPU1 峰值	9.6 万 token	灌满时预填充	解码	暗号
12.8 万、2048、14 GB（原配置）	96,686 MiB	43.2 s	12.8 万，47.9 s	34.1 tok/s	对
12.8 万、1024、14 GB	93,578 MiB	53.4 s	12.8 万，59.4 s	37.7 tok/s	对
19.2 万、1024、14 GB	94,112 MiB	53.1 s	19.3 万，98.4 s	35.0 tok/s	对
25.6 万、1024、14 GB	94,966 MiB	53.1 s	25.9 万，140.0 s	37.7 tok/s	对
19.2 万、2048、10 GB	93,596 MiB	42.9 s	19.3 万，79.7 s	30.9 tok/s	对
19.2 万、2048、12 GB（采用）	**95,604 MiB**	**42.6 s**	**19.3 万，79.4 s**	**33.6 tok/s**	对
19.2 / 25.6 万、2048、14 GB	加载时 GPU1 显存不够（计算缓冲区要 6.7 / 8.6 GiB）				

对拍：同一段 9.6 万 token 的提示词，只改上下文大小时输出逐 token 相同（ubatch 1024 下 12.8 万、19.2 万、25.6 万三者 KL 为 0），采用的配置对原配置 KL 5e-6；ubatch 1024 对 2048 是 0.0066，和只换 ubatch 时的差别一致（§5.5）。19.3 万 token 的提示词在 19.2 万和 25.6 万两种配置上的 32 个 token 相同，KL 6e-4。三种长度都能从开头找回暗号。

表 15 | 上线前后的实际解码，取自 server 的解码统计（每 100 个 token 一个窗口）。两段时间的负载不同：前一段是 Hermes，后一段是 Hermes 和 Codex。

	12.8 万、14 GB	19.2 万、12 GB
时段（2026-10-27）	11:50–14:56	15:25–16:28
窗口数	547	617
每 token 中位数	31.1 ms（32.2 tok/s）	32.1 ms（31.2 tok/s）
最慢的 10%	32.8 ms	33.7 ms
专家缓存命中率	72.2%	69.8%
每 token 上传的专家	15.0	约 16
每轮预填充（0.5–3K token）	2–4 s	2–4 s

上线后的结果见表 15。缓存槽位从约 780 个减到约 670 个，命中率降了约 2.5 个百分点，每 token 多上传约 1 个专家，解码慢约 3%；每轮的预填充不变。另一个代价在最坏情况：崩溃后没有存档、或压缩后重建上下文时，19.3 万 token 从头预填充要约 80 s，12.8 万时约 48 s。

9 换用 1017 权重

DeepSeek 在 10 月 17 日发布了 DeepSeek-V4.1-Flash-1017，称为一次小更新：模型卡上 HLE（不用工具）从 36.8 提到 42.6，Terminal-Bench 4.0 从 31.2 提到 42.3，DeepSWE 从 74.2 提到 77.0，完成任务所需的时间和输出长度也更短，推荐 temperature 1.0、top_p 1.0 [5]。DeepSeek 开源的推理框架 DeepInfer [6] 默认用的就是它。1017 的 `config.json`、`tokenizer.json` 和 `encoding/encoding.py` 与原版逐字节相同，96,085 个张量的名字、类型、形状和所在分片也相同，只换了权重。

现成的路都走不通。fork 公开的转换脚本只支持 V4，没有 Engram，导出 DSpark 侧车的

选项只认 V4 的架构名；mxxm-t 转换 V4.1 用的脚本没有公开。网上的 1017 GGUF 专家都是 K-quant 或 IQ 量化，不是 MXFP4。NVIDIA 的 NVFP4 版本基于原版，专家按 W4A4 设计，每个权重 4.5 bit（比 MXFP4 大约 6%），只支持 vLLM 和 SGLang [7]。

我们写了一个以现有 GGUF 为模板的转换脚本（`tools/convert/v41convert.py`）：每个输出文件照抄模板的文件头，元数据、张量名、类型、形状和偏移都不变，每个张量的数据按表 16 的规则从 HF 权重转换。Q8_0 和 MXFP4 用 ctypes 调用 fork 编译出的 `ggml_quantize_chunk`，舍入和 fork 完全相同。

表 16 | 1017 各类张量的转换规则，与 mxxm-t 转换原版的做法相同。

HF → GGUF	张量	精度
FP4（每字节两个），每 32 个一个 E8M0 → MXFP4	路由专家，按专家号拼成一个张量	不变，只重排半字节
FP8，32 × 32 块 E8M0 → Q8_0	注意力、共享专家、索引器的 q_b、Engram 的 wkv、草稿模型的 fc	反量化后量化
FP8，每 32 个一个 E8M0 → MXFP4	两张 Engram 表（各 3.84 亿行 × 256）	有损（§2.3）
BF16 → F32	各种 norm	不变
照抄	嵌入、输出、路由门、压缩器、索引器投影（BF16）；sink、超连接系数、路由偏置（F32）	不变

规则先在原版上验证：用 HTTP Range 请求远程读取原版权重，不下载整份，68 种张量各取首、中、尾几块，转换结果和 mxxm-t 的 GGUF 逐字节相同；模板里 1,084 个张量全部找到了来源，没用到的 309 个是视觉部分和多模态用的 `gate.bias_vl`。然后转换 1017：475 GiB 权重下载约 90 分钟，48 个分片的 sha256 都已核对；6 个进程并行，按块写到固定偏移，读写都不进页缓存，免得挤掉 server 正在用的 Engram；可以边下边转，专家等 285.6 GiB 用了 3.8 分钟，两张 Engram 表 97.6 GiB 用了 4.6 分钟。转换后核对了 13 个文件的文件头和大小，并随机抽 400 块重新转换，都与写入的一致。

表 17 | 原版与 1017 的对比：同一程序与配置，19.2 万上下文、12 GB 专家缓存。PPL 和 KLD 用 14 段、每段 1,024 token 的语料；这里的 KLD 衡量的是权重的变化，不是数值误差。

	原版	1017
PPL	6.07	5.86（−3.4% ± 0.5%）
对原版的 KLD / top-1 一致率	—	0.075 / 87.7%
GPU1 显存峰值	95,604 MiB	95,604 MiB
9.6 万 token 预填充	42.6 s	42.8 s
19.3 万 token 预填充	79.4 s	79.8 s
满上下文解码*	33.6 tok/s	34.2 tok/s
19.3 万 token 后找回开头的暗号	对	对

* 生成 256 个 token，开了 `ignore_eos`，只适合互相比较（§8）。

结果见表 17。1017 的 PPL 低 3.4%；如果转换有错（例如专家排错），PPL 会成倍上升，而

不是下降。架构和格式都没变，显存和速度与原版相同。生产在 10-27 切换，采样随之改为 top_p 1.0；旧模型存下的槽位对新权重无效，切换时不读回。1017 的 DSpark 侧车也一起转换好了。

10 结论、局限与未来方向

在一台双卡工作站上，Abyss 以原生 MXFP4 精度、19.2 万上下文运行 DeepSeek-V4.1-Flash（10-27 起为 1017 版）：只追加的多轮会话里单流解码约 38 tok/s（9.6 万上下文；生产上真实会话平均 36.8 tok/s）；4.8 万 token 提示词的预填充约 23 s，多轮会话每轮 2.3–3.5 s。收益主要来自五类工作：顺着模型结构减少计算（CED 感知预填充、稀疏索引器、解码注意力只取 top-k），按热度安排数据所在的层级（专家缓存与准入），让 CPU、DMA 和 GPU 同时工作（未命中分流、大批次分流、大批次用缓存），减少重复读取（末段一遍算完），以及把每一块空出来的显存都给专家缓存（计算图的两处浪费、索引器分块、GPU0 的空余显存）。每一项都先剖析、后实现，并用分档的数值底线和对官方实现的对拍确认正确性。

局限。 解码器专家的读取最终受内存带宽和 PCIe 限制：每个未命中的专家都要从双通道内存读一遍，上传的那部分还要经过 x8 的 PCIe（每个约 0.7 ms），而且一层接一层串行；预填充的大批次也已接近「每层专家读一遍」的下限。GPU1 四天里开机出了四次问题（§6.3），PCIe 固定为 Gen5 之后，开机后要先确认两张卡都认到了。显存几乎用满：两张卡的峰值余量只有约 1.5 和 2.0 GiB，再加长上下文或开 DSpark 都得先把专家缓存减回去（§4.5）。server 只有一个槽位，重启时只存它，其余对话要整段重算（§6.4）。Engram 表从官方的 FP8 降到了 MXFP4，我们没有直接测过影响（§2.3）。请求文字里的控制 token 字样在输入侧已按官方规则编码（§6.6），输出侧还没有处理：模型自己用普通 token 写出 `</think>` 或工具调用的标签时，解析器仍按文字识别。DSpark 投机解码和每 token 少用一个专家都测过，没有采用（§4.7）。

未来方向。 前四项来自对 DeepSeek 开源的生产推理框架 DeepInfer [6] 的对照以及外部的实测，其余来自本机的剖析：

- **稀疏预填充注意力。**索引器按查询分块打分已经做了（§4.5）。DeepInfer 的预填充注意力还按下标表只取 SWA 窗口和 top-512 行；fork 仍为每个用索引器的层生成稠密掩码，让 flash attention 带着掩码扫过全部压缩位置。照此改写能省掉掩码，也许能收回分块让大段预填充慢的 4–8%。这项不引入近似。
- **解码器有界重放。** DeepInfer 在生产上预填充时，解码器层只算整段提示词的最后 140 个 token，SWA 截在这一段之内，是近似；分块预填充时完全落在这之前的块只留 1 个占位 token。我们的 CED 是精确的级联（第 20 层 2,414 个 token），实测智能体每轮第 22–38 层每层仍要算约 1,000 个 token（§5.6）。改成有界重放后，要用到的专家和要算的 token 都会少，能省多少要先量，`0015` 和 `--checkpoint-min-step 32768` 这两个折中也可以去掉；需要先用 KLD 对拍近似带来的差别。

- **用重放代替检查点。** DeepInfer 的前缀在任意 128 对齐的位置都能接上，缺失的 SWA 状态靠重算最后 128 个 token 补回。这样任何分叉最多多算 128 个 token，但要改 fork 的 DSV4 缓存管理，工作量最大。
- **DSpark 投机解码。** 4 张 RTX PRO 6000 上的 vLLM 单流开启后，聊天从 83.6 提到 146.8 tok/s，智能体从 82.2 提到 130.5 tok/s，每步接受 2.1–2.8 个草稿 token [9]；DeepInfer 按置信度只验证一部分草稿，验证的 token 少约 30%。本机试过（§4.7）：验证时多出的未命中和缩小的专家缓存抵掉了收益，还和 CED 冲突。要重新评估，得让草稿模型只处理提示词末段的 128 个位置，并先腾出约 10 GB 显存。
- 合并 GPU 上的小内核：解码时每 token 约 2,900 个内核、14.6 ms，占四成多，平均每个只有约 5 μs。
- **重启不丢会话。** 把内存 prompt 缓存里的各段对话连同检查点存盘、启动后读回，重启后的第一个请求不再整段重算（§6.4）。
- **解码时先发命中。** 让 GPU1 先算缓存命中的专家，再等 CPU 算的那部分，两边同时进行，估计快 2–3%（§4.6）。
- **淘汰策略。** 导出解码时每个 token 选的专家，离线比较 LRU、按热度淘汰和理论最优（Belady）的命中率；命中率每高 1 个百分点，解码约快 2%。
- **预填充时 GPU0 边传边算。** 上传暂存区做成两份，让上传和计算流水起来，估计每轮省 0.2–0.4 s；GPU0 只剩约 1.5 GB 余量，可能放不下。
- 与方案 A（3.0 bpw 专家、vLLM 双卡张量并行，预计 55–75 tok/s）做一次同机对比，量化速度与精度的取舍。

参考资料

[1] DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression. 2026. https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash

[2] mxxm-t. mx-llama.cpp 与 DeepSeek-V4.1-Flash 的 MXFP4 GGUF. https://huggingface.co/mxxm-t/DeepSeek-V4.1-Flash-GGUF

[3] diffbot. DeepSeek-V4.1-Flash-EXL3-3bpw-2x-RTX-PRO-6000. https://huggingface.co/diffbot/DeepSeek-V4.1-Flash-EXL3-3bpw-2x-RTX-PRO-6000

[4] Strata：三层分工的本地 MoE 推理. https://github.com/Niko1221/Strata

[5] DeepSeek-AI. DeepSeek-V4.1-Flash-1017. 2026. https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash-1017

[6] DeepSeek-AI. DeepInfer 及其部署问答（`doc/inference_deployment_qa.md`）. 2026. https://github.com/deepseek-ai/deepinfer

[7] NVIDIA. DeepSeek-V4.1-Flash-NVFP4. 2026. https://huggingface.co/nvidia/DeepSeek-V4.1-Flash-NVFP4

[8] DeepSeek-AI. DeepSeek Recipe. 2026. https://github.com/deepseek-ai/deepseek-recipe

[9] Infatoshi. DeepSeek-V4.1-Flash-1017-MXFP4-uncensored，模型卡中的 DSpark 消融. 2026. https://huggingface.co/Infatoshi/DeepSeek-V4.1-Flash-1017-MXFP4-uncensored

A 补丁清单

Abyss 的全部改动以补丁形式保存在仓库的 `mx-llama.cpp/` 目录，基于 mx-llama.cpp 的提交 `c4d12e2`，按序号依次应用。

补丁	作用	开关
0001	server 的专家命中统计	`ABYSS_EXPERT_STATS`
0002	V4.1 的工具调用标签（DSML，标签名带前导空格）	—
0003	CED 感知预填充：解码器层只算可见 token 的 FFN	`ABYSS_CED_PREFILL=1`
0004	Engram 整批预取	`ABYSS_ENGRAM_PREFETCH=8`
0005	CED 感知预填充：注意力也只算子集	同 0003
0006	V4.1 稀疏注意力索引器	`ABYSS_DSV41_INDEXER=1`
0007	`llama-abyss-dump`：导出计算图张量用于对拍	—
0008	单 token 解码时注意力只取 top-k 行	`ABYSS_DSV41_DECODE_GATHER=1`
0009	专家缓存未命中按热度分给上传和 CPU	`ABYSS_MOE_CPU_MISS=1` `ABYSS_MOE_ADMIT=0.3`
0010	修 CED 行映射未分配；未命中分流的全零槽位重复	—
0011	修候选块输入未分配	—
0012	prompt 缓存按能直接用上的 token 数挑选	—
0013	修调度器估算显存时清掉手动指定的后端	—
0014	预填充大批次分流到 CPU	`ABYSS_MOE_BIG_SPLIT=1`
0015	提示词末段一遍算完；CED 余量跟 `n_rs_seq`	`ABYSS_CKPT_TAIL_MERGE=2700` `ABYSS_CED_ALIGN=1`
0016	解码统计：每 N 个解码 token 打印一行每 token 耗时、命中率、上传和 CPU 计算的专家数、缺页次数	`ABYSS_DECODE_STATS=100`
0017	槽位被只共享开头的请求占用前先存进内存缓存；检查点删减间距单独设置	`ABYSS_SLOT_SAVE_LOST` `ABYSS_CKPT_THIN_STEP=8192`
0018	对话提示词按官方 Recipe 的规则编码：只有模板写的控制 token 按 ID 编码	`ABYSS_PLAIN_INPUT=1`
0019	计算图显存：全零张量用到时才分配，没有查询行的层不算查询；可记录每次分配	`GGML_ALLOC_TRACE=1`
0020	预填充的索引器打分和注意力按查询分块	`ABYSS_DSV41_IDX_CHUNK=512`
0021	llama-bench 从环境变量读专家缓存大小	`LLAMA_ARG_MOE_CACHE_MIB`
0022	预填充大批次的统计：每批的专家数和其中缓存已有的	`ABYSS_MOE_BIG_STATS=1`
0023	预填充大批次里缓存已有的专家在缓存所在的卡上算	`ABYSS_MOE_BIG_CACHE=1`

B 复现

生产配置的启动命令如下。需要打过全部补丁的编译，内存在 BIOS 里设 EXPO I + DDR5-5200，命令里的两个 GGML 环境变量是必需的。权重是用 `v41convert.py` 从官方 1017 转换的 GGUF（§9），模板是仓库里的 `deepseek-v4.1.jinja`；仓库的 `restart-server.sh` 还带上

了槽位存取和解码统计：

```
GGML_CUDA_Q8_1_CACHE=0 GGML_CUDA_MMQ_PREC=q8 GOMP_SPINCOUNT=100000000 \
ABYSS_CED_PREFILL=1 ABYSS_ENGRAM_PREFETCH=8 ABYSS_DSV41_INDEXER=1 \
ABYSS_DSV41_IDX_CHUNK=512 ABYSS_DSV41_DECODE_GATHER=1 \
ABYSS_MOE_CPU_MISS=1 ABYSS_MOE_ADMIT=0.3 ABYSS_MOE_BIG_SPLIT=1 \
ABYSS_MOE_BIG_CACHE=1 ABYSS_PLAIN_INPUT=1 \
ABYSS_CKPT_TAIL_MERGE=2700 ABYSS_CED_ALIGN=1 ABYSS_CKPT_THIN_STEP=8192 \
llama-server -m DeepSeek-V4.1-Flash-1017-MXFP4-00001-of-00012.gguf \
  -ngl 99 -sm layer -ts 12/29 -fa 1 -lm dio -c 196608 -b 65536 -ub 2048 \
  -ot "blk\.(2[2-9]|3[0-9])\.ffn_(gate|up|down)_exps\.=CPU" \
  -ot "blk\.12\.ffn_(gate|up)_exps\.=CUDA0" \
  -t 16 -np 1 --moe-cache-mib 20500 --checkpoint-min-step 32768 \
  --jinja --chat-template-file deepseek-v4.1.jinja --no-reasoning-preserve \
  --temp 1.0 --top-p 1.0 --top-k 0 --min-p 0
```

评估脚本在仓库的 `tools/eval/`，复制到机器上运行，运行前要先停 server：

脚本	用途
`kl.sh`	存基准 logits，比较 KLD
`bench.sh`	解码与预填充测速
`prefill-warm.sh`	server 热缓存预填充
`profile-prefill.sh`、`profile-decode.sh`	nsys 剖析预填充、解码
`dump-indexer.sh`、`indexer_oracle.py`	稀疏索引器的单层对拍
`decode-gather.sh`、`decode_attn_oracle.py`	解码注意力的 float64 对拍
`engram-cold.sh`、`engram-share.sh`	Engram 冷读开销与占比
`m2-cpu-miss.sh`	未命中分流的上传比例扫描
`agent-turns.sh`	多轮智能体会话
`decode_modes.sh`	只追加的长会话里测解码，`CAPTURE=1` 剖析
`prefill-sizes.sh`	几种长度新提示词的预填充
`ctx-size.sh`	各上下文配置的显存峰值、长提示词预填充和找回暗号
`slot-takeover.sh`	槽位被只共享开头的请求占用后，主对话还能复用多少
`template_check.py`	对拍对话模板和官方 `encoding.py`
`special_check.py`	逐 token 对拍对话提示词的编码和官方 Recipe 的规则
`align_sweep.py`	调用长度除以 2,048 的各种余数下的边界扫描
`experts5.sh`	每 token 5 个专家对 6 个的 KLD 和解码速度
`dspark.sh`、`dspark_code.py`	DSpark 投机解码的速度、接受率和显存
`alloc-trace.sh`、`alloc_peak.py`	计算缓冲区峰值时有哪些张量
`idx-chunk.sh`	索引器分块的对拍、显存和长提示词
`cache-decode.sh`、`gpu0-experts.sh`	不同专家缓存大小和张量放置下的解码
`admit-sweep.sh`、`threads-sweep.sh`	上传比例、CPU 线程数的扫描
`quick-knobs.sh`	CUDA graph、图优化和分块大小
`profile-prod.sh`、`nsys_layer.py`	用生产设置剖析解码和一轮预填充，打印一层的时间线
`big-coverage.sh`、`big-cache.sh`	大批次的缓存覆盖率；`0023` 开关前后的对比
`std-bench.sh`	llama-bench 和 AIPerf 的标准口径

转换脚本在 `tools/convert/`：`v41convert.py verify` 用原版权重抽样对比模板，`convert` 转换，`check` 核对输出。

C 术语表

按主题分组；同一个词在不同语境下含义不同的（如「解码器层」和「解码」、server 的槽位和专家缓存的槽位），在条目里分别说明。

模型结构

MoE（混合专家） 每层有许多个前馈网络（专家），每个 token 只由路由器挑出其中几个来算。V4.1-Flash 每层 384 个路由专家，每个 token 选 6 个，另有 1 个共享专家每个 token 都算。

路由器（gate） 每层给 384 个专家打分并选出 top-6 的小矩阵；它选中的专家号决定了这一步要读哪些专家权重。

CED（编码器-解码器结构） V4.1 把 40 层分成前 20 层因果编码器（第 0–19 层）和后 20 层解码器（第 20–39 层）。第 20 层用编码器的输出生成所有解码器层共享的全局 KV；解码器层自己的隐状态只能通过 128 个 token 的滑动窗口被其他 token 看到。所以提示词里较早的 token 不必算完解码器层，这是 CED 感知预填充的依据。

编码器层、解码器层 分别指第 0–19 层和第 20–39 层，是模型结构上的划分；和下面推理过程里的「解码」（逐个生成 token）不是一回事。

滑动窗口注意力（SWA） 每个 token 只看自己和前面 127 个位置的 KV（共 128 个）。

压缩注意力、压缩比 把每 2 个或每 1 个 token 的 KV 压成一行，长程注意力只看压缩后的行。第 0–1 层只有滑动窗口，第 2–19 层压缩比为 2，第 20–39 层为 1。压缩 KV 只在第 2、8、14、20 层（KV 源层）写入，其后的层复用。

全局 KV 第 20 层的压缩器生成、被全部解码器层共享的压缩 KV。

稀疏索引器（lightning indexer） 用 32 个头、128 维、FP4 的查询和键给所有压缩位置打分，为每个查询挑出 top-512 个位置；注意力只看这些位置和滑动窗口，共约 640 行。第 2、8、14、20、24、28、32、36 层是索引源层，其余层沿用最近索引源的结果。

候选块 第 20 层按每 8 个压缩位置一块挑出 2,048 块，第 24–36 层只在这些块里重新选 top-512。

超连接（hc） V4 系列对残差连接的扩展：隐状态保留 4 路，每层注意力和前馈网络的前后按学到的系数把它们混合、写回。解码时它带来不少小内核。

Engram V4.1 的查表模块：第 1、14 层按最近 2–4 个 token 的 n-gram 哈希，从两张各约 3.84 亿行的大表里取行。表很大（官方 FP8 约 189 GiB），但每个 token 只读几十行，所以可以放在 NVMe 上。

DSpark V4.1 自带的约 14B 草稿模型，用于投机解码：3 层，每层 128 个专家选 3 个，一次出 5 个 token，读主模型第 37–39 层的隐状态，并用置信度头预测每个草稿 token 被接受的概率。本机试过，暂不采用（§4.7）。

数值格式与量化

MXFP4 4 bit 浮点（E2M1），每 32 个元素共享一个 8 bit 的 2 的幂缩放（E8M0），平均 4.25 bit/权重。V4.1 路由专家的原生格式。

FP8、BF16、F16、F32 8、16、16、32 位浮点。官方 checkpoint 的稠密权重和 Engram 表是 FP8；KV cache 用 F16。

Q8_0 llama.cpp 的 8 bit 整数格式，每 32 个元素一个 F16 缩放，平均 8.5 bit。GGUF 里的稠密权重是这种格式。

W4A8、W4A4 矩阵乘的精度：权重 4 bit，激活量化到 8 bit 或 4 bit。官方参考实现是 W4A8；fork 在 Blackwell 上的预填充默认是 W4A4，我们用 `GGML_CUDA_MMQ_PREC=q8` 改回 8 bit。

bpw 每个权重平均占的位数。方案 A 把专家从约 4.25 bpw 压到 3.0 bpw。

EXL3 ExLlamaV3 的量化格式，方案 A 的公开配方用它。

GGUF llama.cpp 的模型文件格式；本报告起初用 mxxm-t 转换的 MXFP4 GGUF，10-27 起用我们按它的格式自己转换的 1017 版，都是 12 个分片、383 GiB。

推理过程

预填充（prefill） 一次处理整段提示词，写入 KV 并算出第一个 token 的分布。很多 token 一起算，主要受算力和读权重的总量限制。

解码（decode） 每次生成一个 token。每一步都要把用到的权重读一遍，主要受带宽限制。

tok/s 每秒 token 数：预填充指每秒处理的提示词 token，解码指每秒生成的 token。

KV cache 已处理位置的键和值，后面的 token 做注意力时读取。V4.1 的 KV 很小：每个位置只有 1 个 512 维的头，又经过压缩。

llama_decode 调用、ubatch server 把要处理的 token 组成一次 `llama_decode` 调用（上限由 `-b` 决定，本报告用 65536）；一次调用再切成最多 2,048 个 token（`-ub`）的 ubatch，每个 ubatch 把计算图从头到尾跑一遍。

检查点（checkpoint） DeepSeek V4 的缓存不能回退到中间位置。server 在一些位置保存那一刻的完整状态（滑动窗口和压缩器状态），之后的请求如果在这之后才和缓存分叉，就从检查点恢复，不必从头算。

prompt 缓存 server 把处理过的提示词状态（连同检查点）留在内存里，下一个请求前缀相同时直接复用。

server 的槽位（slot） server 同时服务的会话数（`-np`），本报告用 1 个；和专家缓存的槽位无关。

回退环（n_rs_seq） 为投机解码准备的少量回退能力，可以撤销最近几个 token；没开投机解码时为 0，缓存只能退回检查点。

投机解码 用小模型先猜几个 token，再让大模型一次验证，猜对的部分直接接受。

对话模板（chat template） 把多轮消息、工具定义和工具结果拼成模型看到的 token 序列的规则。V4.1 的官方规则在 `encoding/encoding.py`，server 用 Jinja 模板实现，必须逐字一致（§6.5）。

控制 token 对话模板里标记角色和结构的特殊 token，如 `<｜User｜>`、`<｜Assistant｜>`、`</think>`。请求文字里出现同样的字样时，要按普通文字编码（§6.6）。

temperature、top_p 采样参数：temperature 调整概率分布的尖锐程度，top_p 只在累计概率达到 p 的候选里抽取。1017 推荐两者都取 1.0，即按模型原本的分布采样。

有界重放（bounded replay） DeepInfer 的做法：需要重建滑动窗口的状态时，只重算最后 128 个 token，注意力窗口也截在这一段之内，得到近似的 SWA KV；解码器层的预填充也只算末尾 140 个 token。

temperature 0（贪心） 每步都取概率最大的 token，同样的输入一定得到同样的输出，适合对拍。

Abyss 调度器

方案 A、方案 B 方案 A 把专家量化到 3.0 bpw，用 vLLM 双卡张量并行（TP2）；方案 B 保留原生 MXFP4，用 mx-llama.cpp 按层切分，是本报告的主线。

层切分 `-sm layer -ts 12/29`：GPU0 放第 0–11 层整层，GPU1 放第 12–39 层的稠密部分和第 12–21 层的专家，第 22–39 层的专家放主机内存。

锁页内存（CUDA_Host） 不会被操作系统换出、GPU 可以直接用 DMA 读取的主机内存；第 22–39 层的专家（121 GiB）放在这里。

专家缓存、bank、槽位 GPU1 上划出一块显存（bank），分成槽位，每个放一个专家：12.8 万上下文时 14 GB、约 780 个，19.2 万时先是 12 GB、约 670 个，10-28 起 20.5 GB、约 1,140 个；每层有一张从专家号到槽位号的映射表。

命中、未命中、命中率 token 选中的专家已在缓存里叫命中，否则叫未命中；命中率按「token × 专家」计。实际会话里约 65–79%。

LRU 最近最少使用：缓存满了先换出最久没用过的专家。

准入、热度 未命中的专家不是都放进缓存：按衰减热度（每层每步乘 0.99）挑出一定比例（10-28 起为 0.3）上传进缓存，其余由 CPU 算。

未命中分流（0009） 解码时把未命中的专家分成两路同时处理：一部分上传到 GPU1 的缓存里算，其余由 CPU 直接用主机内存里的权重算，每个专家只从内存读一遍。

大批次分流（0014） 预填充时超过 32 个 token 的批次，把分到 token 少的专家交给 CPU，其余照常上传到 GPU0。

CED 感知预填充（0003、0005） 解码器层只对后面的 token 还能通过滑动窗口看到的那些 token 计算前馈网络和注意力，KV 照常全部写入。

大批次用缓存（0023） 预填充的大批次里，缓存已有的专家直接在 GPU1 上用槽位算，其余照旧分给 CPU 和上传到 GPU0，三路同时进行。

索引器分块（0020） 预填充时稀疏索引器的打分和注意力每次只算一段查询（生产用 512 个），计算缓冲区不再随「ubatch × 上下文」整块变大。

末段一遍算完（0015） 让提示词最后一段不再被 server 拆成几次调用，并让一次调用的最后一个 ubatch 是满的，解码器层的专家尽量只读一遍。

计算图、split ggml 把一次前向计算表示成计算图，再按算子所在的设备切成几段（split），每段在一个后端（CPU、GPU0 或 GPU1）上执行，段与段之间拷贝数据、同步。

MMQ、MMVQ CUDA 上的量化矩阵乘内核：MMQ 用于超过 8 个 token 的批次（预填充），MMVQ 用于 1–8 个 token（解码）。

CUDA graph 把一串内核录成一次提交，减少逐个启动的开销。

flash attention 不显式存出完整注意力矩阵、分块计算注意力的内核。

评估方法

对拍 同一段输入下，把改动后的输出和改动前或参照实现逐项比较。

KLD（KL 散度） 两个配置在每个位置上输出的概率分布之差，再取平均；0 表示完全相同。本报告用 llama.cpp 的 `llama-perplexity` 计算。

数值底线 不改变语义的改动（换内核、换 ubatch 大小、换计算设备）也会带来的 KLD。只经过解码器层时约 0.0005，经过编码器层时 0.005–0.01（§3.1）。

top-1 一致率 两个配置下概率最大的 token 相同的位置所占的比例。

top-20 KL 只用 server 返回的前 20 个候选的概率估算的 KL，用来在单 token 的解码路径上对拍。

PPL（困惑度） 模型对一段文本的平均「意外程度」，越低越好。

参照实现（oracle） 我们照官方 `inference/model.py` 写的 PyTorch 实现，用同样的输入逐层重算，检查 fork 的每一步。

nsys NVIDIA Nsight Systems，记录 GPU 内核、拷贝和 CUDA 调用的时间线，用来剖析每个 token 的时间花在哪。

llama-bench、AIPerf llama.cpp 自带的基准程序（pp 测预填充、tg 测生成）和 NVIDIA 的推理服务压测工具（测首 token 时间和每 token 时间），本报告用来给出标准口径（§3.4）。

stressapptest Google 的内存压力测试工具，反复读写大块内存并校验，用来确认内存提频后没有错误。

硬件与系统

PCIe 5.0 x8、Gen3/4/5 显卡和主机之间的链路：第几代决定每条通道的速率，x8 是 8 条通道。Gen5 x8 实测约 28.9 GB/s；开机时链路训练失败会降到 Gen4 或 Gen3。

P2P 两张显卡之间直接拷贝，本机经过 CPU 转发，单向约 27.7 GB/s。

MT/s、双通道 内存每秒传输次数和通道数，两者决定理论带宽：双通道 3600 MT/s 约 57.6 GB/s，5200 MT/s 约 83.2 GB/s。

2DPC、双 rank 每个内存通道插两条内存；每条内存有两组可以分别寻址的芯片。两者都会让内存控制器难以跑高频率。

EXPO、XMP 存在内存条 SPD 芯片里的超频配置（频率、时序、电压），BIOS 可以一键载入；EXPO 是 AMD 的，XMP 是 Intel 的。

UCLK 内存控制器的时钟。和内存时钟 1:1 同频时延迟最低、带宽最好；1:2 时控制器只跑内存时钟的一半，延迟更高，实际带宽也会打折扣。

页缓存、mmap、MADV_WILLNEED Linux 把读过的文件内容留在内存里（页缓存）；mmap 把文件映射到内存地址，访问时按需读入；`MADV_WILLNEED` 提前告诉内核哪些页马上要用，让它预读。

Max-Q RTX PRO 6000 的 300 W 版本，功耗上限低于 600 W 的工作站版。

Codex、MCP Codex 是 OpenAI 的编程智能体命令行工具，本报告用它作为实际负载的客户端；MCP（Model Context Protocol）是给智能体接入外部工具的协议。